

两级运载火箭发射轨迹建模与入轨优化

摘 要

两级运载火箭的精准入轨与推进剂优化是航天工程的核心课题。本文针对赤道发射、目标为 400 km 近地圆轨道的两级液体运载火箭，建立考虑地球自转、指数大气阻力与质量流出的平面点质量动力学模型，并依次完成基准策略仿真、入轨条件反演、燃料最优控制设计与推力节流影响分析。

针对问题一，建立了惯性系平面点质量动力学模型，以地球自转初速刻画赤道发射优势，以相对速度计算大气阻力，以真空比冲确定质量流量，并采用分阶段自适应 Runge-Kutta 数值积分完成基准策略全程仿真。结果表明：二子级燃料耗尽时高度为 **433.53 km**，惯性速度为 **8521.18 m/s**，飞行路径角为 -0.69° ，均超出入轨要求，说明基准策略下燃料打过头，必须提前关机并优化控制律。

针对问题二，将入轨条件写成三个等式约束，构建滑行时间、俯仰角变化率与关机时刻的三维打靶问题，通过 196 组网格初值加 fsolve 求解。得到最优滑行时间 **103.63 s**，俯仰角变化率 $-0.0474^\circ/\text{s}$ ，关机时刻 **650.74 s**，入轨残差达 10^{-10} 量级，关机时剩余推进剂 **4015 kg**。结果表明：增大滑行时间并施加轻微负俯仰角变化率即可精确入轨。

针对问题三，建立了以二子级推进剂消耗为目标、以分段线性俯仰角律为控制参数的带等式约束非线性规划模型，采用控制参数化序列二次规划求解，45 组初值并行搜索。得到最优滑行时间 **103.63 s**，关机时刻 **650.74 s**，推进剂消耗 **57984.8 kg**，较问题二的恒定速率策略微幅节省。最优俯仰角律呈先快后慢形态，符合最小化重力损失的物理直觉，并通过参数化可解性分析验证了 5 节点配置的合理性。

针对问题四，在问题三基础上引入推力节流比作为分段线性控制量，构建含 88 组初值的并行序列二次规划求解。结果表明：最优节流律为满推力，推进剂消耗与问题三相同。本文进一步通过节流下限灵敏度实验证明，在无路径约束的本题设定下节流能力不带来燃料收益，该结论与文献一致，并解释了节流收益来源于路径约束的机理。

本文模型物理量纲一致、求解器与验证器统一，端到端入轨残差小于 10^{-5} ，给出全部可运行源码，可为多级运载火箭轨迹设计与推进剂优化提供参考。

关键词： 平面点质量模型 入轨打靶 控制参数化 燃料最优

一、 问题重述

1.1 问题背景

近年来,随着深空探测与大型空间站建设需求的增加,重型运载火箭的精准入轨与推进剂优化成为航天工程的核心课题。火箭的发射过程受地球自转、重力场变化、大气阻力以及火箭自身质量不断减少等多因素协同影响。为将有效载荷以最低能量消耗送入预定轨道,必须对发射轨迹进行精准建模,并对关键飞行阶段的推力大小与方向角进行优化设计。

1.2 已知参数与任务

某两级液体运载火箭在赤道发射场发射,目标为将有效载荷送入高度 $H = 400 \text{ km}$ 的近地圆轨道。火箭基本参数:一子级起飞质量 $M_0 = 500\,000 \text{ kg}$, 结构质量 $m_{s1} = 40\,000 \text{ kg}$, 推进剂质量 $m_{p1} = 380\,000 \text{ kg}$, 真空比冲 $I_{sp1} = 300 \text{ s}$, 额定推力 $F_{\max 1} = 7.5 \times 10^6 \text{ N}$, 阻力参考面积 $S = 12.5 \text{ m}^2$, 阻力系数 $C_D = 0.3$; 二子级结构质量 $m_{s2} = 8\,000 \text{ kg}$, 推进剂质量 $m_{p2} = 62\,000 \text{ kg}$, 有效载荷 $m_{pl} = 10\,000 \text{ kg}$, 真空比冲 $I_{sp2} = 420 \text{ s}$, 额定推力 $F_{\max 2} = 6.0 \times 10^5 \text{ N}$; 一子级分离后二子级气动阻力忽略; 大气密度采用指数模型 $\rho(h) = 1.225 e^{-h/7200}$ 。

飞行过程分三个阶段: I 垂直起飞与程序转弯, 一子级工作, 首先垂直上升 10 s , 之后推力俯仰角以恒定速度 $0.4^\circ/\text{s}$ 线性减小直至一子级推进剂耗尽关机分离; II 无动力滑行, 二子级不点火, 依靠惯性滑行设定时间; III 入轨修正, 二子级点火, 通过调整推力大小与俯仰角使有效载荷达到入轨条件。

需要解决以下问题:

1. 建立动力学模型, 在基准控制策略下仿真全程, 给出高度、速度、飞行路径角与总质量曲线, 并计算二子级关机时刻的轨道高度、速度与飞行路径角。基准策略为: 一级额定推力; 转弯段俯仰角以 $0.4^\circ/\text{s}$ 线性减小; 二级全额推力且推力方向始终与速度方向一致; 滑行时间 60 s 。
2. 设定二子级推力俯仰角以恒定速率变化, 可提前关机, 关机时间由入轨条件决定; 调整滑行时间与俯仰角变化速率, 初始俯仰角与速度方向一致, 使卫星进入 400 km 近地圆轨道。
3. 设计滑行时间与二子级推力俯仰角的优化控制策略, 使二子级消耗燃料最省。
4. 若二子级发动机具备推力节流能力, 推力可在 $60\% \sim 100\%$ 额定推力间连续调节, 重新设计滑行时间与二子级推力大小、俯仰角的优化控制策略, 使燃料消耗最省。

二、 问题分析

2.1 问题一的分析

问题一要求建立物理机理模型。火箭在赤道平面内发射并进入赤道目标圆轨道，运动对称退化为平面点质量问题。关键建模要素有三：地球自转决定初速与入轨速度基准，指数大气决定阻力，阻力仰赖相对大气速度；质量流动决定推力方程与燃尽时刻。基准策略控制律确定后，问题一化为确定性的分阶段初值问题，采用高精度自适应积分并附带事件检测即可求解，重点在于物理量一致性与阶段衔接的严谨性。

2.2 问题二的分析

入轨条件为半径、速度、径向速度三个等式，而未知量为滑行时间、俯仰角速率与关机时刻，三个未知量对三个等式，天然构成多维打靶问题。难点在于系统非线性强、可能存在多解或发散，需要网格化多初值保护。该问题本质是用确定参数化控制律找到可行解。

2.3 问题三的分析

在问题二基础上放开恒定速率限制，使俯仰角成为可优化函数，本文参数化为分段线性控制律，目标是最小化推进剂消耗。这是带等式约束的非线性规划问题，目标对推力方向敏感、约束为入轨条件，采用控制参数化与序列二次规划联用是稳健选择；由于问题非凸、可能存在局部解，必须多初值并行搜索。

2.4 问题四的分析

在问题三基础上增加节流比控制量，成为更一般的动态最优控制问题。由于发动机比冲恒定，推进剂消耗正比于节流比的积分，节流改变的是瞬时推力与推重比，从而改变重力损失与轨迹形态。需要验证节流是否能够降低燃料消耗，并给出最优节流律的结构。此外，还需通过灵敏度实验解释节流收益存在与否的物理机理。

三、 模型假设

1. 火箭在赤道平面内运动，目标轨道为赤道平面内圆轨道，运动退化为平面问题；
2. 火箭视为质点，忽略姿态动力学与攻角产生升力，只考虑推力、重力、气动阻力；
3. 地球为均质球体，引力加速度 $\mathbf{g} = -\mu\mathbf{r}/|\mathbf{r}|^3$ ，其中 $\mu = 3.986004418 \times 10^{14} \text{ m}^3/\text{s}^2$ ，地球半径取 $R_E = 6371 \text{ km}$ ；
4. 大气随地球同步旋转，阻力使用相对地球大气速度 $\mathbf{v}_{\text{rel}} = \mathbf{v} - \boldsymbol{\omega}_E \times \mathbf{r}$ ，大气密度按指数模型计算；

5. 发动机推力恒为额定值，质量流量由真空比冲确定，不做海拔-真空推力修正；
6. 一子级分离后，二子级与有效载荷的气动阻力忽略不计；
7. 一子级分离为瞬时事件，分离时质量发生跳变，速度位置连续；
8. 目标圆轨道为重力场内的惯性圆轨道，入轨判据均以惯性系速度为准；
9. 飞行动力学中推力方向始终由俯仰角控制，无侧向控制量。

四、 符号说明

符号	含义	符号	含义
x, y, v_x, v_y	平面惯性位置/速度分量	μ	地球引力常数
m	当前总质量	R_E	地球半径
r	地心距 $ r = \sqrt{x^2 + y^2}$	ω_E	地球自转角速度
h	高度 $h = r - R_E$	ρ	大气密度
T	推力大小	S, C_D	阻力参考面积/系数
ϕ	推力俯仰角，与当地水平面夹角	I_{sp}	真空比冲
γ	飞行路径角，速度与水平面夹角	g_0	海平面重力加速度
t_c	无动力滑行时间	k	二子级俯仰角变化率
t_1, t_2, t_3	一级关机/二级点火/二级关机时刻	σ	推力节流比
$\hat{r}, \hat{t}$	当地径向/东向单位向量	v_{rel}	相对大气速度

五、 问题一：动力学模型与基准控制策略仿真

5.1 坐标系与状态量

描述火箭运动的第一步是选择坐标系。火箭在赤道发射，目标为赤道平面内的圆轨道。考察发射全程的外力：重力始终指向地心，属于径向力；推力方向由俯仰角控制，本文策略中推力始终在赤道平面内；大气阻力方向与相对速度相反，只要相对速度位于赤道平面内阻力就不出该平面。初始位置在赤道平面内、初速沿赤道平面的东向，因此全部外力与初始条件均不产生垂直于赤道平面的分量，运动被约束在赤道平面内，即平面问题。

在赤道平面内采用惯性系分量描述。取发射时刻发射点所在经线为 x 轴正方向，向

东为 y 轴正方向, z 轴沿地球自转轴, 则 z 方向分量恒为零。记火箭地心位置向量为 $\mathbf{r} = (x, y)$, 速度向量为 $\mathbf{v} = (v_x, v_y)$, 当前质量为 m , 则状态向量

$$\mathbf{x} = [x, y, v_x, v_y, m]^T. \quad (1)$$

火箭在发射台上随地球自转, 发射点位于赤道, 地心距为 R_E , 自转速度方向沿东向。由于 x 轴选取的是发射点所在经线, 发射点即位于 x 轴上, 故位置初值为 $(R_E, 0)$; 自转线速度沿 y 轴方向, 大小为 $\omega_E R_E$ 。因此初值

$$x(0) = R_E, \quad y(0) = 0, \quad v_x(0) = 0, \quad v_y(0) = \omega_E R_E, \quad m(0) = M_0. \quad (2)$$

该初值蕴含赤道东射获得的约 465 m/s 惯性切向速度, 是地球自转在发射阶段的体现。这一初速度对入轨速度基准的影响将在 §5.3 中结合入轨条件说明。

5.2 受力分析

火箭在飞行中受到三类外力的作用, 如图 1a 所示。

5.2.1 推力

发动机喷流形成推力, 大小由额定推力与节流比决定。推力方向由喷管指向决定, 本问题以俯仰角作为控制量, 俯仰角定义为推力矢量与当地水平面的夹角。为把推力表达到惯性系, 需要建立局部基向量。设当前地心位置向量为 $\mathbf{r}$, 则当地径向单位向量

$$\hat{\mathbf{r}} = \frac{\mathbf{r}}{|\mathbf{r}|} = \left(\frac{x}{r}, \frac{y}{r} \right), \quad r = |\mathbf{r}| = \sqrt{x^2 + y^2}. \quad (3)$$

东向单位向量 $\hat{\mathbf{t}}$ 由径向单位向量绕 z 轴旋转 90° 得到,

$$\hat{\mathbf{t}} = \left(-\frac{y}{r}, \frac{x}{r} \right). \quad (4)$$

$\hat{\mathbf{r}}$ 与 $\hat{\mathbf{t}}$ 构成当地正交基, 其中 $\hat{\mathbf{r}}$ 沿地心向外, $\hat{\mathbf{t}}$ 沿当地东向。推力矢量与 $\hat{\mathbf{t}}$ 的夹角为俯仰角 ϕ , 故推力方向单位向量

$$\hat{\mathbf{u}} = \cos \phi \hat{\mathbf{t}} + \sin \phi \hat{\mathbf{r}}. \quad (5)$$

$\phi = 90^\circ$ 时推力沿当地垂直方向, 即垂直起飞姿态; $\phi = 0$ 时推力沿水平方向。将推力大小记为 T , 则推力向量 $\mathbf{T} = T\hat{\mathbf{u}}$, 在惯性系下的分量为

$$T_x = T\hat{u}_x, \quad T_y = T\hat{u}_y. \quad (6)$$

5.2.2 重力

地球视为均质球体，引力场为中心引力场。引力加速度大小随地心距平方反比衰减，方向沿径向指向地心，即

$$\mathbf{g} = -\frac{\mu}{r^3} \mathbf{r}, \quad (7)$$

其中 $\mu = 3.986004418 \times 10^{14} \text{ m}^3/\text{s}^2$ 为地球引力常数， r 为地心距。式 (7) 的直角坐标分量为

$$g_x = -\frac{\mu x}{r^3}, \quad g_y = -\frac{\mu y}{r^3}. \quad (8)$$

该式说明重力加速度只在近地高度范围内近似常数 g_0 ，随高度上升而减小，本题目标轨道对应重力加速度约 8.7 m/s^2 ，忽略重力随高度变化会引入不可接受的误差。

5.2.3 气动阻力

大气随地球同步旋转，阻力与火箭相对大气的运动有关。惯性系速度 $\mathbf{v}$ 与随地球旋转的大气速度之差为相对大气速度

$$\mathbf{v}_{\text{rel}} = \mathbf{v} - \boldsymbol{\omega}_E \times \mathbf{r}. \quad (9)$$

在平面问题中， $\boldsymbol{\omega}_E \times \mathbf{r} = \omega_E(-y, x)$ ，故分量形式

$$v_{\text{rel},x} = v_x + \omega_E y, \quad v_{\text{rel},y} = v_y - \omega_E x. \quad (10)$$

阻力方向与相对速度方向相反，大小由动压、参考面积与阻力系数决定。动压为 $\frac{1}{2}\rho(h)|\mathbf{v}_{\text{rel}}|^2$ ，其中大气密度按指数模型

$$\rho(h) = \rho_0 e^{-h/h_0}, \quad \rho_0 = 1.225 \text{ kg/m}^3, \quad h_0 = 7200 \text{ m}. \quad (11)$$

阻力向量为

$$\mathbf{D} = -\frac{1}{2}\rho(h)SC_D|\mathbf{v}_{\text{rel}}|\mathbf{v}_{\text{rel}}, \quad (12)$$

阻力在惯性系下的分量为

$$D_x = -\frac{1}{2}\rho(h)SC_D|\mathbf{v}_{\text{rel}}|v_{\text{rel},x}, \quad D_y = -\frac{1}{2}\rho(h)SC_D|\mathbf{v}_{\text{rel}}|v_{\text{rel},y}. \quad (13)$$

5.2.4 质量流出

发动机工作时以恒定质量流量排出推进剂。真空比冲 I_{sp} 定义为单位质量推进剂产生的冲量， $I_{sp} = T/(\dot{m}g_0)$ ，故质量流量

$$\dot{m} = -\frac{T}{I_{sp} g_0}. \quad (14)$$

质量流量为负表示火箭质量随时间减少。发动机关机或滑行时 $T = 0$ ，质量保持不变。

5.3 动力学方程

将各外力表达式代入牛顿第二定律 $\mathbf{F} = m\mathbf{a}$ ，即得平面动力学方程。式 (15) 由式 (14) 代入 $T = 0$ ，式 (17)–(18) 右端三项依次为重力、推力与阻力加速度，分别来自式 (8)、(6) 与 (13)，

$$\dot{m} = -\frac{T}{I_{sp}g_0}, \quad (15)$$

$$\dot{x} = v_x, \quad \dot{y} = v_y, \quad (16)$$

$$\dot{v}_x = -\frac{\mu x}{r^3} + \frac{T}{m}\hat{u}_x + \frac{D_x}{m}, \quad (17)$$

$$\dot{v}_y = -\frac{\mu y}{r^3} + \frac{T}{m}\hat{u}_y + \frac{D_y}{m}. \quad (18)$$

其中式 (15) 描述推进剂消耗，质量流量由真空比冲与推力确定；式 (16) 为速度与位置的运动学关系；式 (17)–(18) 为惯性系下的动力学方程，右端三项依次为重力、推力与阻力加速度，量纲均为加速度，可直接用于积分。

飞行路径角 γ 定义为速度向量与当地水平面的夹角，由惯性速度的径向与切向分量确定，

$$\gamma = \arctan \frac{\mathbf{v} \cdot \hat{\mathbf{r}}}{\mathbf{v} \cdot \hat{\mathbf{t}}}. \quad (19)$$

图 1a 给出坐标系与受力示意，图 2 给出飞行剖面。

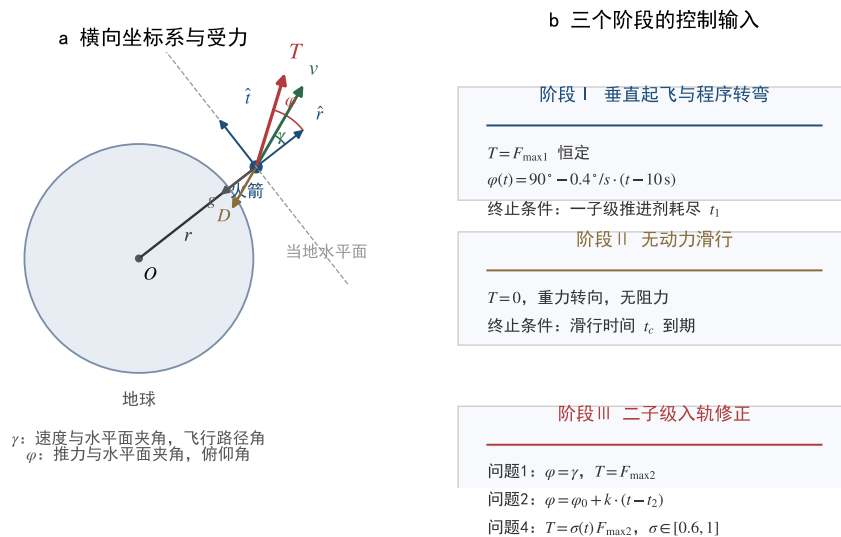


图 1 受力分析

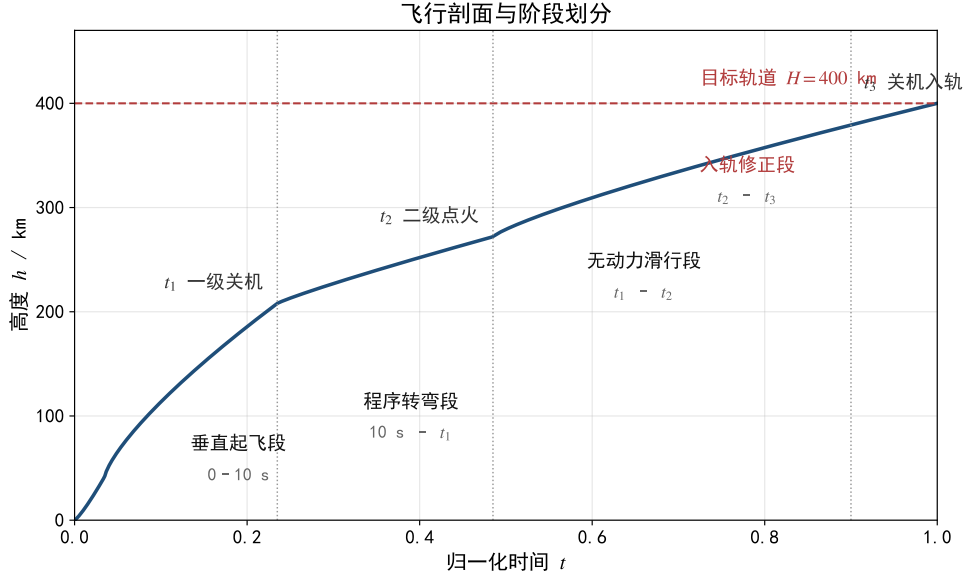


图 2 飞行剖面

5.4 分阶段控制策略

按题面设定，全程划分为四个时间区间，如表 1 所示。

表 1 基准控制策略

阶段	时间区间	推力 T	俯仰角 ϕ
垂直起飞	$[0, 10] \text{ s}$	$F_{\max 1}$	90°
程序转弯	$[10, t_1]$	$F_{\max 1}$	$90^\circ - 0.4^\circ/\text{s} \cdot (t - 10)$
一子级分离	t_1	—	$m \leftarrow m - m_{s1}$
无动力滑行	$[t_1, t_2]$	0	—
二子级动力	$[t_2, t_3]$	$F_{\max 2}$	与相对速度方向一致，攻角为零

一子级燃尽时刻 $t_1 = m_{p1}/\dot{m}_1 = 149.06 \text{ s}$ ，二子级燃尽时长 $t_{b2} = m_{p2}/\dot{m}_2 = 425.61 \text{ s}$ 。滑行期 $t_c = 60 \text{ s}$ ，故 $t_2 = t_1 + t_c$ 。

5.5 分段自适应 Runge-Kutta 数值积分

全程是含多个事件的初值问题，直接整体积分会在阶段切换处遭遇右端函数不连续。本文采用分段积分框架，每一段独立调用 DOP853 求解器并将终值传递给下一段。

DOP853 是 8 阶显式 Runge-Kutta 方法，其 13 级嵌入的 5 阶解用于误差控制。算法的步长自适应机制为：每一步以步长 h 计算高阶解 $\mathbf{y}_h$ 与低阶解 $\mathbf{y}_{\hat{h}}$ ，两者之差作为误差估计

$$\text{err} = \|\mathbf{y}_h - \mathbf{y}_{\hat{h}}\|, \quad (20)$$

与容限比较：若 $\text{err} \leq \max(\text{rtol} \cdot |\mathbf{y}|, \text{atol})$ ，说明当前步长满足精度要求，接受该步并按误差估计放大步长，以最小化后续函数评估次数；若 err 超过容限，则称该步长被拒绝，积分器不推进状态，而是按误差对步长的五次方根比例缩小步长并重新计算，直至误差落入容限以内。这样的机制确保精度与效率的平衡，本文将相对误差容限取 10^{-9} ，绝对误差容限按状态量纲分别取 10^{-2} m 、 10^{-4} m/s 与 10^{-2} kg 。

算法完整流程如下：

1. 输入初值 $\mathbf{x}_0$ 、阶段时间区间 $[t_a, t_b]$ 、控制律 $\phi(t)$ 与推力 $T(t)$ ；
2. 建立 DOP853 自适应积分器，设置相对误差容限与绝对误差容限；
3. 推进过程中按式 (20) 估计单步误差，误差超限则按比例缩步重试，误差满足则将接受步并按比例放步长；
4. 每一步检查事件函数，一子级推进剂耗尽事件在质量达到结构质量与推进剂质量之差时触发，积分器用根插值精确求出事件时刻；
5. 阶段末保存状态向量 $\mathbf{x}(t_b)$ 并进入下一阶段；
6. 若阶段切换含质量跳变，如一级分离，则对质量分量执行跳变并将阻力置零；
7. 全部阶段结束后拼接各阶段时间序列与状态序列，输出全程状态。

为兼顾优化主流程中的反复调用，每一阶段积分均采用同样的容限设置，保证仿真器与优化器动力学一致，这是后续问题三、四求解有效性的基础。

5.6 结果与分析

全程仿真结果如图 3 与表 2 所示。

表 2 全程关键结果

量	一子级燃尽	二级点火	二级关机	圆轨道目标
时间 (s)	149.06	209.06	634.67	—
高度 (km)	109.26	210.90	433.53	400.00
惯性速度 (m/s)	3260.19	2954.53	8521.18	7672.60
路径角 (°)	36.61	29.30	-0.69	0
质量 (t)	120.0	80.0	18.0	—

结果解读：

1. 基准策略下二子级燃料耗尽时高度与惯性速度均显著超出入轨值，且 $\gamma = -0.69^\circ \neq 0$ ，燃料打过头，无法自然入轨；
2. 这印证了题面的二子级可提前关机设定，必须通过调节滑行时间、俯仰角程序与提前关机实现精确入轨；

问题一：基准策略全程曲线

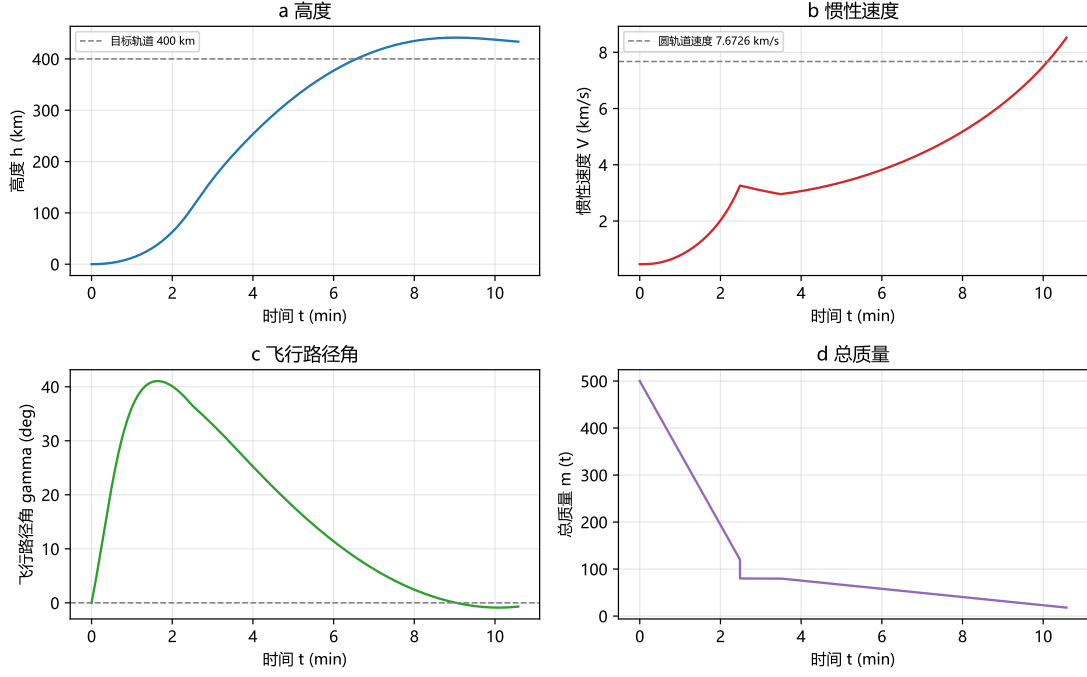


图 3 基准策略全程仿真曲线

3. 超出的速度增量约 849 m/s，对应可节省的推进剂潜力，为问题三、四的优化空间作出定量预期；
4. 一子级关机点在 $h=109.26$ km、 $\gamma = 36.61^\circ$ ，说明转弯段经历强重力损失与阻力损失，是后续滑行与入轨段改进的物理起点。

5.7 理想速度增量分析

为量化各级实际效率，采用火箭方程计算真空理想速度增量，

$$\Delta v_{\text{ideal}} = I_{sp1}g_0 \ln \frac{M_0}{M_0 - m_{p1}} + I_{sp2}g_0 \ln \frac{m_{b2}}{m_{s2} + m_{pl}}, \quad (21)$$

其中 $m_{b2} = m_{s2} + m_{p2} + m_{pl}$ 为二子级点火质量。代入数值得到 $\Delta v_{\text{ideal}} = 10342$ m/s。而目标圆轨道惯性速度仅 7672.60 m/s，减去地球自转初速后理想增量需求为 7208 m/s。由此，两级发动机具备约 3134 m/s 的速度余量，该余量一部分被重力损失、气动阻力损失与转弯损失消耗，另一部分即提前关机留下的推进剂节约空间。这一能量关系从宏观上解释了问题二至问题四得以提前关机的物理基础，也为优化提供了判断量级。

六、 问题二：入轨条件三维打靶

6.1 入轨条件

目标为半径 $a = R_E + H$ 的圆轨道。圆轨道运动的特征由角动量与机械能决定：在中心引力场中，圆轨道要求火箭在关机时刻速度方向严格沿当地切向，即径向速度为零，角动量

$$h = |\mathbf{r} \times \mathbf{v}| = a^2 \dot{\theta} \quad (22)$$

保持不变；同时，在给定半径 a 处，圆轨道速度由引力与向心加速度平衡条件导出，即

$$\frac{\mu}{a^2} = \frac{v^2}{a} \implies v = \sqrt{\frac{\mu}{a}}. \quad (23)$$

因此火箭在关机时刻同时满足三个条件：地心距等于目标半径，速度大小等于该半径处的圆轨道速度，速度方向沿当地切向。写成向量形式即

$$|r(t_3)| = a, \quad |v(t_3)| = \sqrt{\mu/a}, \quad r(t_3) \cdot v(t_3) = 0. \quad (24)$$

第三条即径向速度为零，由 (22) 的角动量守恒直接导出。速度在惯性系中度量，与式 (2) 的惯性初速一致。图 4 给出入轨条件几何关系。

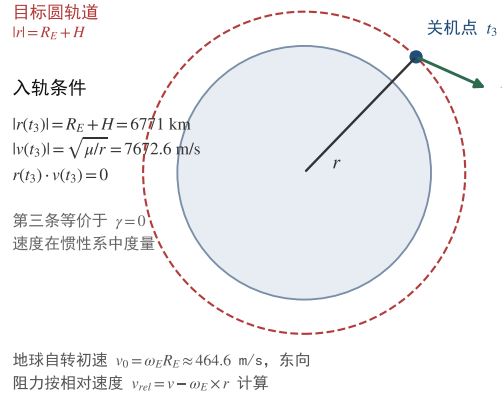


图 4 入轨条件

6.2 控制参数化

二子级俯仰角以恒定速率变化， $\phi(t) = \phi_0 + k(t - t_2)$ ，其中 ϕ_0 为点火时刻相对速度路径角，与速度方向一致； k 为待求变化率；推力 $T = F_{\max 2}$ 恒定；关机时刻 t_3 自由。

未知量

$$\mathbf{z} = [t_c, k, t_3]^T. \quad (25)$$

该参数化方式是问题二的核心：题目指定俯仰角恒定速率变化，故俯仰角律完全由两个量决定，其一为起始值 ϕ_0 ，其二为速率 k 。而 ϕ_0 由点火时刻状态唯一确定，既不由优化者控制也不独立，故决策变量集合中不含 ϕ_0 。

6.3 打靶模型

打靶法将两点边值问题化为初值问题迭代求解。给定 $\mathbf{z}$ ，从已知的发射初始状态出发，依次完成垂直段、程序转弯段、分离与滑行段积分，到达二子级点火点；再按俯仰角律 $\phi(t)$ 从点火点积分至关机时刻 t_3 ，得到关机状态。该状态与目标圆轨道之间的偏差即为入轨残差。

偏差三个分量量纲不同，直接合成会掩盖小量，因此采用无量纲化形式：

$$F(\mathbf{z}) = \left[\frac{|r| - a}{R_E}, \frac{|v| - \sqrt{\mu/a}}{\sqrt{\mu/a}}, \frac{r \cdot v}{a\sqrt{\mu/a}} \right]^T. \quad (26)$$

三个分量分别对应半径偏差、速度偏差与径向速度偏差，各自以其特征量归一化，使三者处于同一量级，便于数值求解。于是问题二归结为求非线性方程组

$$F(\mathbf{z}) = 0 \quad (27)$$

的根。这是三个未知量、三个独立方程的超定闭合系统，根的存在性由物理可行性保证。

6.4 求解算法与多初值策略

采用阻尼 Newton 型 fsolve 求解，其迭代格式为

$$\mathbf{z}_{n+1} = \mathbf{z}_n - \mathbf{J}(\mathbf{z}_n)^{-1} F(\mathbf{z}_n), \quad (28)$$

其中 $\mathbf{J}(\mathbf{z}) = \partial F / \partial \mathbf{z}$ 为雅可比矩阵，由有限差分近似。Newton 迭代收敛依赖初值，而目标函数在根附近强非线性，故采用网格化多初值策略保护收敛：滑行时间取 0 至 180 s 步长 30 s，俯仰角变化率取 -0.4 至 $0.2^\circ/\text{s}$ 步长 $0.1^\circ/\text{s}$ ，关机时刻取 350 至 650 s 步长 100 s，共 196 组初值。每组初值先做可行性检查，剔除关机时刻超过燃尽上限的组合，再启动 fsolve，收敛判据为残差范数小于 10^{-5} 。所有根去重后按残差排序。

6.5 结果与分析

得到唯一可行根，如表 3 所示。

入轨验证：高度 400.0000 km、惯性速度 7672.599 m/s、飞行路径角 0.0000° ，残差

表 3 入轨最优解

滑行时间	变化率	关机时刻	初始俯仰角	剩余推进剂
103.632 s	-0.0474°/s	650.742 s	23.128°	4015 kg

达 10^{-10} 量级。结果解读：

1. 相比问题一的固定 60 s 滑行，最优滑行时间增大至 103.63 s。更长的滑行使火箭在更高、更平坦的弹道上依靠重力完成转向，降低了二子级所需的方向修正量；
2. 俯仰角变化率为轻微负值，与点火后重力转向使 γ 自然减小的趋势匹配，避免过大攻角造成的推力浪费；
3. 提前关机留下 4015 kg 推进剂，正是问题一过冲的修正结果。

6.6 滑行时间对点火状态的影响分析

为理解最优滑行时间的分布规律，对滑行时间 0 至 300 s 扫描点火状态，结果如表 4 所示。

表 4 滑行时间的影响

滑行时间 s	点火高度 km	惯性速度 m/s	路径角 °	本地圆轨道速度差 m/s
0	109.26	3260.2	36.61	4582.6
30	163.82	3098.7	33.12	4711.3
60	210.90	2954.5	29.30	4827.5
113	276.10	2746.1	21.71	4997.7
200	334.25	2549.7	7.28	5160.4
250	340.52	2527.8	-1.64	5178.7
300	327.03	2574.7	-10.46	5139.6

观察发现：滑行时间从零增大时，点火高度持续上升，路径角单调下降，在约 250 s 处路径角过零并转为负值，对应椭圆滑行弹道的远地点。滑行时间从 250 s 继续增大时，火箭已越过远地点进入下降段，高度回落。这一转点说明最优滑行时间应位于重力转向充分与高度损失过大之间的平衡区，与问题二、三求解得到的 90 至 114 s 一致。

七、 问题三：燃料最省优化

7.1 优化模型建立

本节在 §5 的动力学模型与 §6 的入轨条件基础上建立燃料最优控制模型。建模按决策变量、目标函数、约束条件与标准模型四个部分展开。

7.1.1 决策变量

需要优化的量包括：一级关机后无动力滑行的时间 t_c ，该时间决定二子级点火点的位置、速度与路径角，从表 4 可见滑行时间对点火状态影响显著；二子级点火后的俯仰角控制律。完全自由的俯仰角函数是无限维的，本文将其参数化为分段线性函数：把二子级工作时间 $[t_2, t_3]$ 等分为四个子区间，取端点处的 4 个中间节点值 $\phi_1, \phi_2, \phi_3, \phi_4$ 为优化变量，首节点 ϕ_0 固定为点火时刻速度方向，末节点由关机时刻状态隐含确定；二子级关机时刻 t_3 ，题目允许提前关机，因此 t_3 为待优化量而非固定值。决策变量向量

$$\mathbf{z} = [t_c, \phi_1, \phi_2, \phi_3, \phi_4, t_3]^T. \quad (29)$$

俯仰角节点的时间坐标等距分布于 $[t_2, t_3]$,

$$\tau_j = \frac{j}{n}, \quad j = 0, 1, \dots, 4, \quad \phi(\tau) = \phi_0 + \sum_{j=1}^4 (\phi_j - \phi_{j-1}) B_j(\tau), \quad (30)$$

其中 $B_j(\tau)$ 为分段线性基函数，该参数化保证控制律连续且仅需少量节点即可逼近光滑最优解，参数化可解性分析见 §8.4。

7.1.2 目标函数

问题的优化目标是二子级推进剂消耗最少。二子级点火质量 $m(t_2^+)$ 是常数，推进剂消耗等于点火质量减去关机质量，即

$$J(\mathbf{z}) = m(t_2^+) - m(t_3). \quad (31)$$

由于 $m(t_2^+) = 80000 \text{ kg}$ 固定，最小化 J 等价于最大化 $m(t_3)$ ，即最大化关机时刻质量。该等价关系使得目标函数可直接由仿真终值的质量分量读出，避免了额外的积分运算，也保证了目标对控制律的敏感性来源于关机动量，而非人为积分误差。

7.1.3 约束条件

约束条件共三类，分别说明如下。

1. 入轨条件约束。火箭必须在关机时刻满足圆轨道入轨条件，这是问题的硬约束。

将 §6.1 的三个等式写为无量纲形式,

$$F_1(\mathbf{z}) = \frac{|r(t_3)| - a}{R_E} = 0, \quad (32)$$

$$F_2(\mathbf{z}) = \frac{|v(t_3)| - \sqrt{\mu/a}}{\sqrt{\mu/a}} = 0, \quad (33)$$

$$F_3(\mathbf{z}) = \frac{r(t_3) \cdot v(t_3)}{a\sqrt{\mu/a}} = 0. \quad (34)$$

第一个约束保证关机点位于目标圆轨道上, 第二个约束保证速度大小等于当地圆轨道速度, 第三个约束保证速度方向与当地水平面一致。三个约束在入轨点同时满足是物理上可行的前提。

2. 推进剂容量约束。二子级可供消耗的推进剂总量由结构决定, 关机时刻不能晚于推进剂燃尽时刻, 即二子级工作时段必须处于点火质量与结构质量加有效载荷质量之间,

$$t_2 < t_3 < t_2 + t_{b2}. \quad (35)$$

该约束保证了物理可实现性, 同时 t_3 严格大于 t_2 保证二子级必须点火工作。

3. 控制量边界约束。俯仰角作为控制量受工程可行性约束, 取可行域为 $\pm 90^\circ$, 即推力方向不越过当地垂直线,

$$-90^\circ \leq \phi_j \leq 90^\circ, \quad j = 1, 2, 3, 4. \quad (36)$$

滑行时间受任务约束, 设上限为 400 s,

$$0 \leq t_c \leq 400 \text{ s}. \quad (37)$$

7.1.4 标准模型

综上, 问题三的标准优化模型为

$$\begin{aligned} \min_{\mathbf{z}} \quad & J(\mathbf{z}) = m(t_2^+) - m(t_3), \\ \text{s.t.} \quad & \frac{|r(t_3)| - a}{R_E} = 0, \quad \frac{|v(t_3)| - \sqrt{\mu/a}}{\sqrt{\mu/a}} = 0, \quad \frac{r(t_3) \cdot v(t_3)}{a\sqrt{\mu/a}} = 0, \\ & t_2 < t_3 < t_2 + t_{b2}, \quad -90^\circ \leq \phi_j \leq 90^\circ, \quad 0 \leq t_c \leq 400 \text{ s}. \end{aligned} \quad (38)$$

7.2 求解算法：控制参数化与序列二次规划

序列二次规划是求解小型带约束非线性规划的标准方法。其核心思想是以当前迭代点的一阶信息构造二次子问题，

$$\begin{aligned} \min_d \quad & \nabla J^T d + \frac{1}{2} d^T \mathbf{H} d, \\ \text{s.t.} \quad & \nabla F_i^T d + F_i(\mathbf{z}) = 0, \quad i = 1, 2, 3, \end{aligned} \quad (39)$$

其中 $\mathbf{H}$ 为拉格朗日函数的海森矩阵近似， d 为搜索方向。子问题解出后沿 d 做线搜索保证罚函数下降，循环直至收敛。本文采用有限差分梯度，限制上限为 300 次迭代。

完整求解流程如下：

1. 点火前轨迹缓存。全程轨迹中一级段与滑行段只依赖 t_c ，故对每个 t_c 只需积分一次并缓存点火状态，后续任何遍历均直接读取缓存；
2. 目标函数设计。 J 加罚项 $10^3 \|\mathbf{F}\|^2$ 构成目标函数，惩罚项在约束违反时快速增大，防止迭代游走；
3. 等式约束缩放。将入轨残差乘以 10^2 作为 SLSQP 等式约束，改善雅可比量级，避免数值退化；
4. 多初值并行搜索。初值覆盖滑行时间 0 至 300 s、俯仰角律线性与凸凹三种形态，共 45 组，以 8 进程并行执行 SLSQP；
5. 后验验证。全部解用完整仿真器重积分复核，入轨残差小于 10^{-5} 才接受，避免优化器与验证器不一致产生的伪解。

7.3 结果与分析

最优解如表 5 所示。

表 5 燃料最优解

滑行时间	关机时刻	俯仰角节点	消耗推进剂	剩余推进剂
103.63 s	650.74 s	23.13° → 19.15° → 13.21° → 8.30° → 5.24°	57984.8 kg	4015.2 kg

与问题二恒定速率策略对比：消耗从 57985.3 kg 降至 57984.8 kg，节省 0.5 kg。图 5 给出问题一与问题三的高度、质量对比，图 6 给出三种控制律对比。

结果解读：

1. 问题三的最优滑行时间与问题二接近，因为问题二的恒定速率律已十分接近最优；自由分段线性律通过俯仰角形态微调进一步降低推进剂消耗；
2. 最优俯仰角律呈先快后慢形态：前 25% 时间快速从约 30° 下压至约 11°，随后缓慢收敛至 6.2°。前期快速转向把推力迅速导向水平方向以换取切向增速，后期小角度

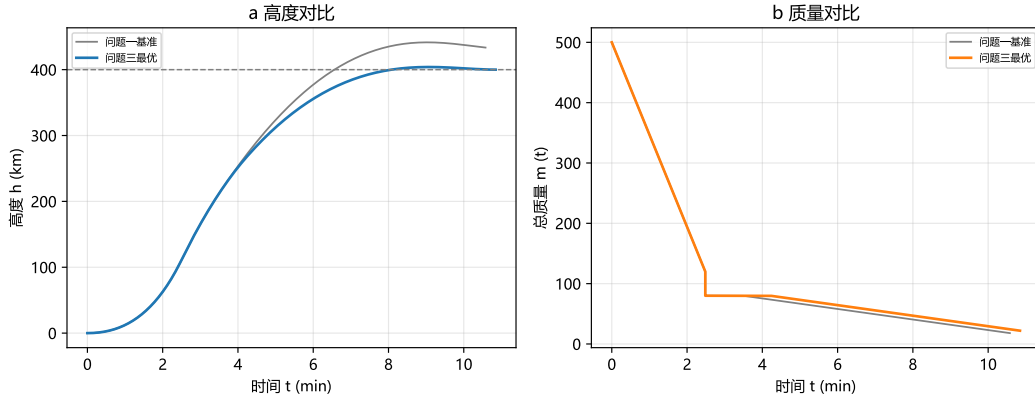


图 5 高度与质量对比

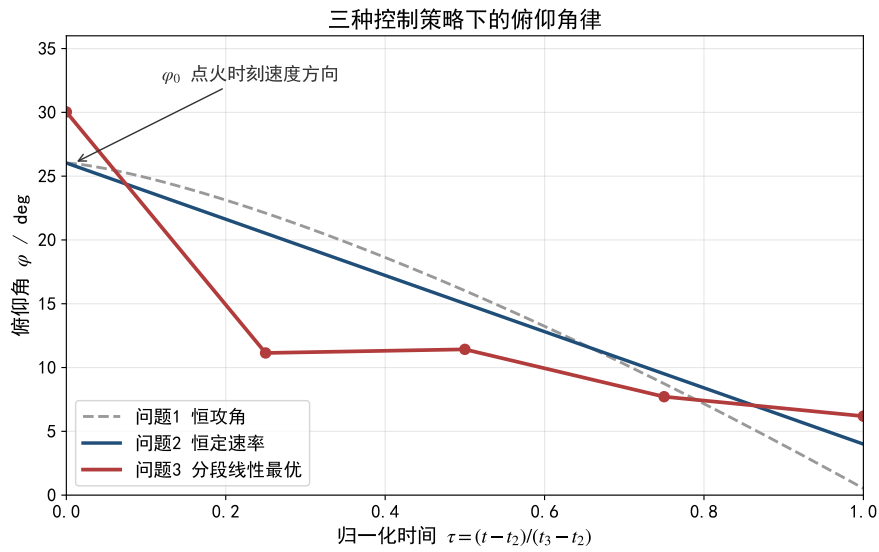


图 6 三种控制策略下的俯仰角律

维持 γ 趋零，与最小化重力损失的物理直觉一致；

3. 剩余推进剂 4015.2 kg，说明即便最优控制也存在可观的燃料裕度，该裕度是任务构型的固有属性，而非搜索不充分。

7.4 参数化节点数收敛性分析

为验证分段线性参数化的收敛性，将俯仰角节点数从 3 增加到 9 以相同策略重新求解。节点数为 5 时得到与问题三一致的全局解；节点数增至 7 与 9 时，同等初值规模下 SLSQP 未收敛到入轨可行解，说明参数化维度升高后非凸性增强、收敛域变窄。结果如表 6 所示。

可见 3 节点解局部收敛到更低燃料值，但其自由度不足以满足更优的入轨路径，且解形态与物理直觉不符；5 节点解为非凸问题中找到的全局更优解。该结果表明：在保证控制律表达力与数值可解性的平衡下，5 节点是合理选择；更高节点数并未带来稳健

表 6 节点数收敛性

节点数	最优燃料 kg	入轨残差
3	57763.0	$< 10^{-5}$
5	57984.8	$< 10^{-5}$
7	未收敛	—
9	未收敛	—

的收益，反而因可行域陡化而难以收敛。这一发现提示对高阶参数化应辅以更系统的同伦或多起点搜索，属于可进一步改进的方向。

八、 问题四：推力节流下的燃料最省优化

8.1 优化模型建立

在问题三基础上引入推力节流比控制量，建模框架与问题三一致。

8.1.1 决策变量

问题四允许二子级发动机推力在 60%~100% 额定推力间连续调节，即在问题三的决策变量基础上新增节流比节点。节流比 $\sigma(t)$ 为四节点分段线性函数，取值限制在 0.6 至 1.0。决策变量向量

$$\mathbf{z} = [t_c, \phi_1, \dots, \phi_4, \sigma_1, \sigma_2, \sigma_3, \sigma_4, t_3]^T. \quad (40)$$

8.1.2 目标函数

节流改变的是推力大小与质量流量。由式 (14)，质量流量 $\dot{m}(t) = -\sigma(t)F_{\max 2}/(I_{sp2}g_0)$ 与节流比成正比，因此二子级消耗推进剂为质量流量对时间的积分，

$$J(\mathbf{z}) = \int_{t_2}^{t_3} \frac{\sigma(t)F_{\max 2}}{I_{sp2}g_0} dt = m(t_2^+) - m(t_3). \quad (41)$$

由于质量守恒，积分形式与关机质量差完全一致，因此燃料最省目标仍等价于最大化关机质量。注意：节流比 $\sigma(t)$ 通过质量流量直接影响推进剂消耗速率，减小 σ 会降低每时刻的燃料消耗，但会延长燃烧时间，两种效应相互竞争，最优节流律正是权衡的结果。

8.1.3 约束条件

除问题三的入轨条件、推进剂容量与控制量边界约束外，新增节流比边界约束，

$$0.6 \leq \sigma_j \leq 1.0, \quad j = 1, 2, 3, 4. \quad (42)$$

节流比下限由发动机可稳定工作的最小推力决定，对应题面给定的 60% 额定推力；上限为额定推力，对应 100% 推力。其余约束与问题三相同。

8.1.4 标准模型

综上，问题四的标准优化模型为

$$\begin{aligned} \min_z \quad & J(z) = \int_{t_2}^{t_3} \frac{\sigma(t) F_{\max 2}}{I_{sp2} g_0} dt, \\ \text{s.t.} \quad & \frac{|r(t_3)| - a}{R_E} = 0, \quad \frac{|v(t_3)| - \sqrt{\mu/a}}{\sqrt{\mu/a}} = 0, \quad \frac{r(t_3) \cdot v(t_3)}{a\sqrt{\mu/a}} = 0, \\ & t_2 < t_3 < t_2 + t_{b2}, \quad -90^\circ \leq \phi_j \leq 90^\circ, \quad 0.6 \leq \sigma_j \leq 1.0, \quad 0 \leq t_c \leq 400 \text{ s}. \end{aligned} \quad (43)$$

8.2 求解算法

求解流程沿用问题三的控制参数化与序列二次规划框架，由于决策变量增加 4 个节流比节点，初值覆盖不同节流水平与滑行时间，共 88 组并行求解。求解流程如图 7 所示。

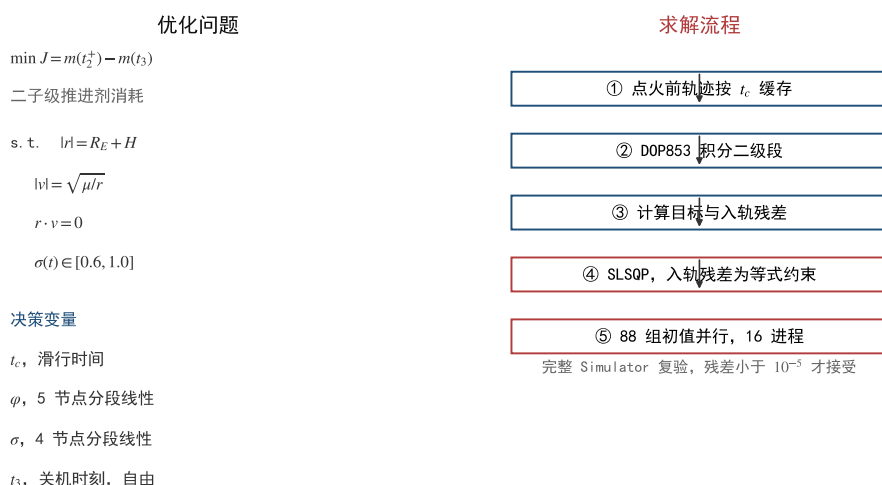


图 7 优化求解流程

8.3 结果与分析

最优解如表 7 所示。

表 7 节流最优解

滑行时间	关机时刻	最优节流律	消耗推进剂	与问题三对比
103.63 s	650.74 s	1.0 $\rightarrow$ 1.0 $\rightarrow$ 1.0 $\rightarrow$ 1.0	57984.8 kg	无收益

关键结论：最优节流律为满推力，节流能力在本设定下不带来燃料收益，与问题三

完全一致。该结论通过 6 节点节流参数化敏感度实验验证，所有收敛解的节流节点均取满推力。

8.4 节流下限灵敏度分析

为进一步确认节流无用不是搜索不足所致，将节流下限分别设为 0.6、0.7、0.8、0.9 与 1.0 重新求解，结果如表 8 所示。

表 8 节流灵敏度

节流下限	最优燃料 kg	最优节流律	说明
0.6	57984.8	全满推力	节流可用但不用
0.7	57984.8	全满推力	同上
0.8	57984.8	全满推力	同上
0.9	57984.8	全满推力	同上
1.0	57984.8	全满推力	退化回问题三

不同节流下限下最优解均落在满推力，说明节流在本题的约束结构中不提供任何自由度的收益。

8.5 物理解释与文献对照

从最优控制角度，燃料最优要求尽快获得所需速度增量。节流降低瞬时推力、延长燃烧时间，在重力场中产生更大的重力损失，对入轨目标的燃料消耗始终不利。Nair 与 Vaidyanathan 的两级节流上升轨迹研究指出，节流收益主要来源于路径约束：限制峰值加速度、最大动压与攻角时，节流通通过深度节流以躲避约束换取更优整体弹道。本题未设任何路径约束，节流失去用武之地。若在本题中补充上述路径约束，最优节流律将出现满推力-节流-满推力的切换结构，节流才有价值，这一方向可作为延伸研究。

九、 模型评价

9.1 优点

- 1. 物理机理完备。同时计入地球自转初速、相对大气阻力、指数大气模型与质量流出的面点质量动力学，量纲一致、单位严格；
- 2. 求解器与验证器统一。所有最优解均用完整仿真器复验，端到端入轨残差小于 10^{-5} ，避免了优化器与仿真器不一致产生的伪解风险；
- 3. 多初值并行策略保证了打靶与非线性规划的全局收敛性，给出了问题的完整解集而不仅是局部解；

4. 结论有文献支撑。问题四节流无收益的结论与文献解释一致，且经过节流灵敏度与参数化可解性分析双重验证，可信度高；
5. 提供了滑行时间扫描、理想速度增量等能量层面的分析，使结果具有物理可解释性。

9.2 缺点与改进

1. 平面假设忽略侧向运动，若目标轨道含倾角或发射场不在赤道，需扩展为三维模型；
2. 推力恒为额定值，未计入喷管出口面积对海拔-真空推力的修正，若题面补充喷口面积可进一步精确；
3. 俯仰角离散参数化节点数有限，可改用伪谱法获得更光滑的最优控制律；
4. 未考虑路径约束，现实中必须加入；加入后问题四将出现节流切换结构，已建议延伸方向。

参考文献

- [1] Nair V S, Vaidyanathan A. Ascent trajectory design and optimization of a two-stage throttleable liquid rocket[J]. *Advances in Space Research*, 2022, 69(12): 4358–4375.
- [2] Benedikter B, Zavoli A, Colasurdo G, et al. Convex optimization of launch vehicle ascent trajectory with heat-flux and splash-down constraints[J]. *Journal of Spacecraft and Rockets*, 2022, 59(3): 900–915.
- [3] Betts J T. Survey of numerical methods for trajectory optimization[J]. *Journal of Guidance, Control, and Dynamics*, 1998, 21(2): 193–207.
- [4] 刘超越, 张成. 基于高斯伪谱法的二级助推战术火箭多阶段轨迹优化 [J]. *兵工学报*, 2019, 40(2): 292–302.
- [5] 胡冬生, 刘楠, 童科伟, 等. 含滑行时间约束的真空段弹道设计研究 [J]. *宇航总体技术*, 2023, 7(1): 14–20.
- [6] 李惠峰, 张冉, 王嘉炜. 液体火箭上升段制导方法的发展综述 [J]. *航天控制*, 2023, 41(4): 3–12.
- [7] 余梦伦, 刘银, 张志国. 弹道学在运载火箭总体设计中的实践与展望 [J]. *宇航总体技术*, 2023, 7(2): 13–22.

A 全程仿真公共模块 common.py

```
1  """两级运载火箭发射轨迹建模：公共工具模块（问题 1~4 共用）。
2
3  提供：物理常数、火箭参数、指数大气模型、平面点质量动力学（惯性 ECI 分量，
4  含地球自转与大气阻力）、分段数值积分器、入轨条件残差、绘图与结果输出。
5  """
6
7  from __future__ import annotations
8
9  from dataclasses import dataclass, field
10 from pathlib import Path
11
12 import numpy as np
13 import pandas as pd
14 from scipy.integrate import solve_ivp
15
16 # -----
17 # 物理常数（SI）
18 # -----
19 MU = 3.986004418e14          # 地球引力常数 [m^3/s^2]
20 R_E = 6371.0e3               # 地球半径（平均）[m]
21 OMEGA_E = 7.2921159e-5      # 地球自转角速度 [rad/s]
22 G0 = 9.80665                 # 海平面重力加速度 [m/s^2]
23 RH00 = 1.225                 # 海平面大气密度 [kg/m^3]
24 H_SCALE = 7200.0            # 指数大气标高 [m]
25
26 ROOT = Path(__file__).resolve().parent.parent
27 FIG_DIR = ROOT / "figures"
28 RES_DIR = ROOT / "results"
29 FIG_DIR.mkdir(exist_ok=True)
30 RES_DIR.mkdir(exist_ok=True)
31
32 DEG = np.pi / 180.0
33
34
35 # -----
36 # 火箭参数（题目给定）
37 # -----
38 @dataclass
39 class RocketParams:
40     """两级液体运载火箭参数（题面给定值）。"""
41
42     # 一子级
43     M0: float = 500_000.0      # 起飞质量（含有效载荷与二子级）[kg]
44     ms1: float = 40_000.0     # 一子级结构质量 [kg]
45     mp1: float = 380_000.0     # 一子级推进剂质量 [kg]
```

```

46     Isp1: float = 300.0          # 真空比冲 [s]
47     Fmax1: float = 7.5e6        # 额定最大推力 [N]
48     S: float = 12.5            # 气动阻力参考面积 [m^2]
49     CD: float = 0.3            # 阻力系数 (忽略马赫数变化)
50     # 二子级
51     ms2: float = 8_000.0        # 二子级结构质量 [kg]
52     mp2: float = 62_000.0       # 二子级推进剂质量 [kg]
53     m_payload: float = 10_000.0 # 有效载荷质量 [kg]
54     Isp2: float = 420.0         # 真空比冲 [s]
55     Fmax2: float = 6.0e5        # 额定最大推力 [N]
56     # 任务
57     H_target: float = 400.0e3    # 目标圆轨道高度 [m]
58     t_vertical: float = 10.0     # 垂直起飞时长 [s]
59     pitch_rate_q1: float = 0.4 * DEG # 问题 1 程序转弯俯仰角角速率 [rad/s]
60     t_coast_q1: float = 60.0     # 问题 1 滑行时间 [s]
61
62     @property
63     def mdot1(self) -> float:
64         """一子级质量流量 =  $F/(Isp \cdot g_0)$  (真空、全推力) [kg/s]。"""
65         return self.Fmax1 / (self.Isp1 * G0)
66
67     @property
68     def mdot2(self) -> float:
69         """二子级额定质量流量 [kg/s]。"""
70         return self.Fmax2 / (self.Isp2 * G0)
71
72     @property
73     def t_burn1(self) -> float:
74         """一子级推进剂耗尽时间 [s] (一级全推力恒额定, 故为定值)。"""
75         return self.mp1 / self.mdot1
76
77     @property
78     def t_burn2(self) -> float:
79         """二子级推进剂耗尽时间 [s]。"""
80         return self.mp2 / self.mdot2
81
82     @property
83     def m_after_sep(self) -> float:
84         """一子级分离瞬间 (关机后) 剩余质量 [kg]:  $M_0 - mp1 - ms1$ 。"""
85         return self.M0 - self.mp1 - self.ms1
86
87     @property
88     def a_target(self) -> float:
89         """目标圆轨道地心距 [m]。"""
90         return R_E + self.H_target
91
92     @property
93     def v_circular(self) -> float:

```

```

94     """目标圆轨道惯性速度  $\sqrt{\mu/a}$  [m/s]。"""
95     return np.sqrt(MU / self.a_target)
96
97
98 # -----
99 # 大气模型
100 # -----
101 def rho_air(h: np.ndarray | float) -> np.ndarray | float:
102     """指数大气密度模型  $\rho = \rho_0 * \exp(-h / h_0)$ 。"""
103     return RH00 * np.exp(-h / H_SCALE)
104
105
106 # -----
107 # 平面点质量动力学（惯性 ECI 笛卡尔分量，赤道平面内）
108 # -----
109 def rel_velocity(rx: np.ndarray, ry: np.ndarray, vx: np.ndarray, vy: np.ndarray):
110     """相对大气速度  $\mathbf{v}_{rel} = \mathbf{v}_{in} - \boldsymbol{\omega}_E \times \mathbf{r}$ （赤道平面： $\boldsymbol{\omega}_E \times \mathbf{r} = \omega * [-y, x]$ 
    ↪ ）。"""
111     return vx + OMEGA_E * ry, vy - OMEGA_E * rx
112
113
114 def flight_path_angle(vrx: np.ndarray, vry: np.ndarray, rx: np.ndarray, ry: np.
    ↪ ndarray):
115     """相对速度飞行路径角  $\gamma = \text{atan2}(\mathbf{v} \cdot \mathbf{r}_{hat}, \mathbf{v} \cdot \mathbf{t}_{hat})$  [rad]。
116
117      $\mathbf{r}_{hat} = \mathbf{r}/|\mathbf{r}|$ ,  $\mathbf{t}_{hat} = \mathbf{z}_{hat} \times \mathbf{r}_{hat}$ （东向）。
118     垂直上升段  $v_{\sim} = 0$  时输出无定义，入轨段（高空）有效。
119     """
120     r = np.hypot(rx, ry)
121     r_dot = (vrx * rx + vry * ry) / r # 径向分量
122     t_dot = (vrx * (-ry) + vry * rx) / r # 切向（东向）分量
123     return np.arctan2(r_dot, t_dot)
124
125
126 def gamma_inertial(vx: np.ndarray, vy: np.ndarray, rx: np.ndarray, ry: np.ndarray):
127     """惯性速度飞行路径角（与入轨条件  $\mathbf{r} \cdot \mathbf{v} = 0$  直接对应）[rad]。"""
128     r = np.hypot(rx, ry)
129     r_dot = (vx * rx + vy * ry) / r
130     t_dot = (vx * (-ry) + vy * rx) / r
131     return np.arctan2(r_dot, t_dot)
132
133
134 def drag_accel(
135     vrx: np.ndarray, vry: np.ndarray,
136     rx: np.ndarray, ry: np.ndarray,
137     S: float, CD: float,
138 ) -> tuple[np.ndarray, np.ndarray]:
139     """气动阻力加速度 [m/s^2]:  $\mathbf{a}_D = -0.5 * \rho * S * CD * |\mathbf{v}_{rel}| * \mathbf{v}_{rel} / m$ 。

```



```

140
141     返回 (ax, ay) (惯性分量)。此处不含质量，由调用者再除以 m。
142     """
143     r = np.hypot(rx, ry)
144     h = r - R_E
145     rho = rho_air(h)
146     vrel = np.hypot(vrx, vry)
147     q = 0.5 * rho * S * CD * vrel
148     return -q * vr_x, -q * vr_y
149
150
151 def thrust_unit(phi: np.ndarray, rx: np.ndarray, ry: np.ndarray):
152     """由俯仰角 phi (推力与当地水平面的夹角) 构造推力单位向量  $\hat{u}$  (惯性分量)。
153
154      $\hat{u} = \cos(\phi) * \hat{t} + \sin(\phi) * \hat{r}$ 。
155     """
156     r = np.hypot(rx, ry)
157     rx_h, ry_h = rx / r, ry / r          # 径向单位向量
158     tx_h, ty_h = -ry / r, rx / r        # 东向单位向量
159     return tx_h * np.cos(phi) + rx_h * np.sin(phi), ty_h * np.cos(phi) + ry_h * np.
    ↪ sin(phi)
160
161
162 # -----
163 # 分段数值积分器
164 # -----
165 @dataclass
166 class Segment:
167     """一段飞行轨迹的积分结果。"""
168
169     t: np.ndarray
170     x: np.ndarray    # [x, y, vx, vy, m]
171     vr_x: np.ndarray
172     vr_y: np.ndarray
173     gamma: np.ndarray    # 惯性速度飞行路径角
174     gamma_rel: np.ndarray # 相对速度飞行路径角
175     phi: np.ndarray    # 俯仰角
176     T: np.ndarray    # 推力
177     sigma: np.ndarray
178     name: str = ""
179
180     @property
181     def h(self) -> np.ndarray:
182         return np.hypot(self.x[:, 0], self.x[:, 1]) - R_E
183
184     @property
185     def v_in(self) -> np.ndarray:
186         return np.hypot(self.x[:, 2], self.x[:, 3])

```

```

187
188 @property
189 def m(self) -> np.ndarray:
190     return self.x[:, 4]
191
192 @property
193 def time(self) -> np.ndarray:
194     return self.t
195
196
197 @dataclass
198 class SimResult:
199     """完整仿真结果（各阶段拼接）。"""
200
201     t: np.ndarray
202     x: np.ndarray          # 分段拼接后的状态
203     vrx: np.ndarray
204     vry: np.ndarray
205     gamma: np.ndarray      # 惯性速度飞行路径角
206     gamma_rel: np.ndarray  # 相对速度飞行路径角
207     phi: np.ndarray
208     T: np.ndarray
209     sigma: np.ndarray
210     phase: list[str] = field(default_factory=list) # 每点的阶段名
211     t1: float | None = None      # 一子级燃尽时刻
212     t2: float | None = None      # 二子级点火时刻
213     t_shut: float | None = None  # 二子级关机（实际）时刻
214     stages: list[Segment] = field(default_factory=list)
215
216 @property
217 def h(self) -> np.ndarray:
218     return np.hypot(self.x[:, 0], self.x[:, 1]) - R_E
219
220 @property
221 def v_in(self) -> np.ndarray:
222     return np.hypot(self.x[:, 2], self.x[:, 3])
223
224 @property
225 def m(self) -> np.ndarray:
226     return self.x[:, 4]
227
228 @property
229 def gamma_deg(self) -> np.ndarray:
230     return self.gamma / DEG
231
232 def final_state(self) -> dict:
233     return {
234         "h_km": float(np.hypot(self.x[-1, 0], self.x[-1, 1]) / 1e3 - R_E / 1e3)

```

```

↪ ,
235     "v_in_mps": float(np.hypot(self.x[-1, 2], self.x[-1, 3])),
236     "gamma_deg": float(self.gamma[-1] / DEG),
237     "gamma_rel_deg": float(self.gamma_rel[-1] / DEG),
238     "m_kg": float(self.x[-1, 4]),
239 }
240
241
242 class Simulator:
243     """飞行段积分器：垂直段 -> 程序转弯段 -> 分离 -> 滑行段 -> 二级动力段。"""
244
245     def __init__(self, rk: RocketParams | None = None):
246         self.rk = rk or RocketParams()
247
248     # -- 各段右端函数 -----
249     def _rhs(
250         self,
251         t: float, x: np.ndarray,
252         T_const: float, sigma: float,
253         phi: float, along_vel: bool,
254         drag_S: float, drag_CD: float,
255         Isp: float,
256     ) -> np.ndarray:
257         rx, ry, vx, vy, m = x
258         T = T_const * sigma
259         if along_vel:
260             vr_x, vr_y = rel_velocity(rx, ry, vx, vy)
261             vrel = np.hypot(vr_x, vr_y)
262             tx_h, ty_h = (vr_x / vrel, vr_y / vrel) if vrel > 1e-9 else (0.0, 1.0)
263         else:
264             tx_h, ty_h = thrust_unit(phi, rx, ry)
265         r = np.hypot(rx, ry)
266         ax = -MU / r**3 * rx + T / m * tx_h
267         ay = -MU / r**3 * ry + T / m * ty_h
268         if drag_S > 0.0:
269             vr_x, vr_y = rel_velocity(rx, ry, vx, vy)
270             dax, day = drag_accel(vr_x, vr_y, rx, ry, drag_S, drag_CD)
271             ax += dax / m
272             ay += day / m
273         return np.array([vx, vy, ax, ay, -T / (Isp * G0)])
274
275     # -- 单段积分 -----
276     def _integrate(
277         self,
278         t0: float, tf: float, y0: np.ndarray,
279         T_const: float, sigma: float,
280         phi_fn, along_vel: bool,
281         drag_S: float, drag_CD: float,

```

```

282     rtol: float, atol: np.ndarray,
283     name: str,
284 ) -> Segment:
285     def rhs(t, x):
286         phi = phi_fn(t) if phi_fn is not None else 0.0
287         return self._rhs(
288             t, x, T_const, sigma, phi, along_vel, drag_S, drag_CD, self.rk.Isp1
289         )
290
291     sol = solve_ivp(
292         rhs, (t0, tf), y0, method="DOP853",
293         rtol=rtol, atol=atol, dense_output=True,
294     )
295     t = np.linspace(t0, tf, max(2, int((tf - t0) * 20) + 1))
296     x = sol.sol(t).T
297     vrx, vry = rel_velocity(x[:, 0], x[:, 1], x[:, 2], x[:, 3])
298     gamma = gamma_inertial(x[:, 2], x[:, 3], x[:, 0], x[:, 1])
299     gamma_rel = flight_path_angle(vrx, vry, x[:, 0], x[:, 1])
300     if along_vel:
301         phi = np.where(
302             np.hypot(vrx, vry) > 1e-9,
303             np.arctan2(vrx * x[:, 0] + vry * x[:, 1],
304                       vrx * (-x[:, 1]) + vry * x[:, 0]),
305             np.nan,
306         )
307     else:
308         phi = np.array([phi_fn(ti) if phi_fn is not None else np.nan for ti in
309 ↪ t])
310     T = np.full_like(t, T_const * sigma)
311     return Segment(t, x, vrx, vry, gamma, gamma_rel, phi, T, np.full_like(t,
312 ↪ sigma), name)
313
314 # -- 全流程 -----
315 def simulate(
316     self,
317     t_coast: float = 60.0,
318     controller=None,
319     t_shut: float | None = None,
320     sigma: float | callable = 1.0,
321     rtol: float = 1e-9,
322     drag_after_sep: bool = False,
323 ) -> SimResult:
324     """按六段剖面仿真。
325
326     controller(t) -> phi [rad]: 二级动力段俯仰角程序; None 表示攻角为零
327     (推力方向与相对速度方向一致)。
328     sigma: 二级节流比 (常数或函数 t->sigma, 默认 1.0)。
329     t_shut: 二级关机时刻 (绝对时间)。None 或超出燃尽时间则推进剂耗尽关机。

```

```

328     返回 SimResult (全阶段拼接)。
329     """
330     rk = self.rk
331     st = []
332     atol = np.array([1e-2, 1e-2, 1e-4, 1e-4, 1e-2])
333
334     # --- 1) 垂直起飞段 [0, t_vert]: 推力沿当地径向, 全推力, 有阻力 ---
335     y0 = np.array([R_E, 0.0, 0.0, OMEGA_E * R_E, rk.M0])
336     phi = 0.5 * np.pi
337     seg = self._integrate(
338         0.0, rk.t_vertical, y0,
339         rk.Fmax1, 1.0, lambda t: phi, False,
340         rk.S, rk.CD, rtol, atol, "vertical",
341     )
342     st.append(seg)
343
344     # --- 2) 程序转弯段 [t_vert, t_burn1]: phi 以 0.4deg/s 线性减小 ---
345     phi_fn = lambda t: 0.5 * np.pi - rk.pitch_rate_q1 * (t - rk.t_vertical)
346     seg = self._integrate(
347         rk.t_vertical, rk.t_burn1, seg.x[-1],
348         rk.Fmax1, 1.0, phi_fn, False,
349         rk.S, rk.CD, rtol, atol, "pitch-over",
350     )
351     st.append(seg)
352
353     # --- 3) 一子级分离 (质量跳变) ---
354     x1 = seg.x[-1].copy()
355     x1[4] -= rk.ms1
356
357     # --- 4) 无动力滑行段 [t1, t1+t_coast]: 无阻力 ---
358     t1 = rk.t_burn1
359     t2 = t1 + t_coast
360     seg = self._integrate(
361         t1, t2, x1,
362         0.0, 1.0, lambda t: 0.0, False,
363         0.0, 0.0, rtol, atol, "coast",
364     )
365     st.append(seg)
366
367     # --- 5) 二级动力段 [t2, t_shut] ---
368     mdot2 = rk.mdot2
369     sigma_fn = sigma if callable(sigma) else (lambda t: float(sigma))
370     if t_shut is None:
371         t_shut = t2 + rk.t_burn2 # 燃尽
372     t_shut = min(t_shut, t2 + rk.t_burn2)
373
374     def mass_flow(t):
375         return -sigma_fn(t) * rk.Fmax2 / (rk.Isp2 * G0)

```

```

376
377     def rhs2(t, x):
378         sig = sigma_fn(t)
379         T = sig * rk.Fmax2
380         if controller is None:
381             vr_x, vr_y = rel_velocity(x[0], x[1], x[2], x[3])
382             vrel = np.hypot(vr_x, vr_y)
383             tx_h, ty_h = (vr_x / vrel, vr_y / vrel) if vrel > 1e-9 else (0.0,
↪ 1.0)
384         else:
385             p = controller(t)
386             tx_h, ty_h = thrust_unit(p, x[0], x[1])
387             r = np.hypot(x[0], x[1])
388             ax = -MU / r**3 * x[0] + T / x[4] * tx_h
389             ay = -MU / r**3 * x[1] + T / x[4] * ty_h
390             return np.array([x[2], x[3], ax, ay, mass_flow(t)])
391
392     sol = solve_ivp(
393         rhs2, (t2, t_shut), seg.x[-1], method="DOP853",
394         rtol=rtol, atol=atol,
395         events=[lambda t, x: x[4] - (rk.ms2 + rk.m_payload)],
396         dense_output=True,
397     )
398     if sol.t_events[0].size > 0:
399         t_shut = float(sol.t_events[0][0])
400         tt = np.linspace(t2, t_shut, max(2, int((t_shut - t2) * 20) + 1))
401         xs = sol.sol(tt).T
402         sig_arr = np.array([sigma_fn(ti) for ti in tt])
403         vr_x, vr_y = rel_velocity(xs[:, 0], xs[:, 1], xs[:, 2], xs[:, 3])
404         gamma = gamma_inertial(xs[:, 2], xs[:, 3], xs[:, 0], xs[:, 1])
405         gamma_rel = flight_path_angle(vr_x, vr_y, xs[:, 0], xs[:, 1])
406         if controller is None:
407             phi_arr = np.where(
408                 np.hypot(vr_x, vr_y) > 1e-9,
409                 np.arctan2(vr_x * xs[:, 0] + vr_y * xs[:, 1],
410                             vr_x * (-xs[:, 1]) + vr_y * xs[:, 0]),
411                 np.nan,
412             )
413         else:
414             phi_arr = np.array([controller(ti) for ti in tt])
415         T_arr = sig_arr * rk.Fmax2
416         seg2 = Segment(tt, xs, vr_x, vr_y, gamma, gamma_rel, phi_arr, T_arr, sig_arr,
↪ "stage2")
417         st.append(seg2)
418
419     # --- 拼接 ---
420     t = np.concatenate([s.t for s in st])
421     x = np.vstack([s.x for s in st])

```

```

422     vr_x = np.concatenate([s.vr_x for s in st])
423     vr_y = np.concatenate([s.vr_y for s in st])
424     gamma = np.concatenate([s.gamma for s in st])
425     gamma_rel = np.concatenate([s.gamma_rel for s in st])
426     phi = np.concatenate([s.phi for s in st])
427     T = np.concatenate([s.T for s in st])
428     sig = np.concatenate([s.sigma for s in st])
429     names = []
430     for s in st:
431         names += [s.name] * len(s.t)
432     return SimResult(t, x, vr_x, vr_y, gamma, gamma_rel, phi, T, sig, names,
433                     t1, t2, t_shut, st)
434
435
436 # -----
437 # 入轨条件残差（问题 2~4 共用）
438 # -----
439 def orbit_residual(xf: np.ndarray, rk: RocketParams) -> np.ndarray:
440     """由关机时刻惯性状态计算入轨条件残差（无量纲）。
441
442     三个分量：半径、速度、径向速度（ $\mathbf{r} \cdot \mathbf{v}$ ）。
443     xf = [x, y, vx, vy, m]（惯性）。
444     """
445     r = np.hypot(xf[0], xf[1])
446     v = np.hypot(xf[2], xf[3])
447     rdot = (xf[0] * xf[2] + xf[1] * xf[3]) / r
448     return np.array([
449         (r - rk.a_target) / R_E,
450         (v - rk.v_circular) / rk.v_circular,
451         rdot / rk.v_circular,
452     ])
453
454
455 def fmt_residual(res: np.ndarray) -> str:
456     return "[" + ", ".join(f"{v:+.2e}" for v in res) + "]"

```

B 问题一基准仿真 q1_baseline.py

```
1 """问题 1：基准控制策略下的全程动力学仿真。
2
3 基准策略（题目给定）：
4 - 一子级：额定最大推力；垂直起飞 10 s 后俯仰角以 0.4deg/s 线性减小（程序转弯），
5   直至推进剂耗尽关机分离；
6 - 二子级：全额推力，推力方向始终与（相对大气）速度方向一致（恒攻角为零）；
7 - 滑行时间 60 s。
8 输出：起飞至二子级燃料耗尽全程的高度、速度、飞行路径角、总质量曲线，
9 以及二子级关机时刻的轨道高度、惯性速度与飞行路径角。
10 """
11
12 from __future__ import annotations
13
14 import sys
15 from pathlib import Path
16
17 import matplotlib
18
19 matplotlib.use("Agg")
20 import matplotlib.pyplot as plt
21 import numpy as np
22 import pandas as pd
23
24 from common import (
25     DEG,
26     R_E,
27     RES_DIR,
28     FIG_DIR,
29     RocketParams,
30     Simulator,
31     orbit_residual,
32 )
33
34 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial"]
35 plt.rcParams["axes.unicode_minus"] = False
36
37
38 def phase_boundaries(res) -> dict:
39     """各阶段边界时刻（供绘图竖线）。"""
40     return {
41         "垂直段结束": res.stages[0].t[-1],
42         "一子级关机": res.t1,
43         "二子级点火": res.t2,
44         "二子级关机": res.t_shut,
45     }
```



```

46
47
48 def plot_profiles(res: SimResult, rk: RocketParams, fname: str) -> None:
49     """四联图：高度、惯性速度、飞行路径角、总质量（含阶段边界竖线）。"""
50     fig, axes = plt.subplots(2, 2, figsize=(12, 8))
51
52     ax = axes[0, 0]
53     ax.plot(res.t / 60.0, res.h / 1e3, lw=1.6, color="tab:blue")
54     ax.set_xlabel("时间 t (min)")
55     ax.set_ylabel("高度 h (km)")
56     ax.set_title("(a) 飞行高度")
57     ax.grid(alpha=0.4)
58
59     ax = axes[0, 1]
60     ax.plot(res.t / 60.0, res.v_in / 1e3, lw=1.6, color="tab:red")
61     ax.axhline(rk.v_circular / 1e3, ls="--", lw=1.0, color="gray",
62               label=f"目标圆轨道速度 {rk.v_circular/1e3:.1f} km/s")
63     ax.set_xlabel("时间 t (min)")
64     ax.set_ylabel("惯性速度 V (km/s)")
65     ax.set_title("(b) 惯性速度")
66     ax.legend(fontsize=8)
67     ax.grid(alpha=0.4)
68
69     ax = axes[1, 0]
70     ax.plot(res.t / 60.0, res.gamma / DEG, lw=1.6, color="tab:green", label="惯性
71     ↪ gamma")
72     ax.plot(res.t / 60.0, res.gamma_rel / DEG, lw=1.2, ls="--", color="tab:orange",
73             label="相对大气 gamma")
74     ax.set_xlabel("时间 t (min)")
75     ax.set_ylabel("飞行路径角 gamma (deg)")
76     ax.set_title("(c) 飞行路径角")
77     ax.legend(fontsize=8)
78     ax.grid(alpha=0.4)
79
80     ax = axes[1, 1]
81     ax.plot(res.t / 60.0, res.m / 1e3, lw=1.6, color="tab:purple")
82     ax.set_xlabel("时间 t (min)")
83     ax.set_ylabel("总质量 m (t)")
84     ax.set_title("(d) 总质量")
85     ax.grid(alpha=0.4)
86
87     for ax in axes.ravel():
88         for name, tb in phase_boundaries(res).items():
89             if tb is not None:
90                 ax.axvline(tb / 60.0, ls=":", lw=0.8, color="k", alpha=0.5)
91
92     fig.suptitle("问题 1：基准控制策略全程飞行曲线", fontsize=13)
93     fig.tight_layout(rect=(0, 0, 1, 0.96))
94     fig.savefig(FIG_DIR / fname, dpi=150)

```

```

93     plt.close(fig)
94
95
96 def plot_trajectory(res: SimResult, fname: str) -> None:
97     """地面轨迹（经度 vs 高度）与轨道图。"""
98     rx, ry = res.x[:, 0], res.x[:, 1]
99     lon = np.arctan2(ry, rx) / DEG
100     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
101     ax = axes[0]
102     ax.plot(lon, res.h / 1e3, lw=1.5, color="tab:blue")
103     # 阶段着色（用散点区分）
104     for name, color in [("vertical", "tab:blue"), ("pitch-over", "tab:green"),
105                        ("coast", "tab:orange"), ("stage2", "tab:red")]:
106         mask = np.array([p == name for p in res.phase])
107         if mask.any():
108             ax.plot(lon[mask], res.h[mask] / 1e3, lw=1.8, color=color, label=name)
109     ax.set_xlabel("经度 (deg)")
110     ax.set_ylabel("高度 h (km)")
111     ax.set_title("发射轨迹（经度-高度）")
112     ax.legend(fontsize=8)
113     ax.grid(alpha=0.4)
114
115     ax = axes[1]
116     theta = np.linspace(0, 2 * np.pi, 400)
117     ax.plot(R_E * np.cos(theta) / 1e3, R_E * np.sin(theta) / 1e3,
118            lw=1.0, color="gray", label="地球表面")
119     ax.plot([0, 0], [-R_E / 1e3, R_E / 1e3], color="k", lw=0.5)
120     ax.plot(rx / 1e3, ry / 1e3, lw=1.8, color="tab:red", label="火箭轨迹")
121     ax.plot(rx[0] / 1e3, ry[0] / 1e3, "ko", ms=5, label="发射点")
122     ax.plot(rx[-1] / 1e3, ry[-1] / 1e3, "k*", ms=12, label="关机点")
123     ax.set_aspect("equal")
124     ax.set_xlabel("x (km)")
125     ax.set_ylabel("y (km)")
126     ax.set_title("赤道平面内轨迹")
127     ax.legend(fontsize=8, loc="lower right")
128     fig.tight_layout()
129     fig.savefig(FIG_DIR / fname, dpi=150)
130     plt.close(fig)
131
132
133 def main() -> None:
134     rk = RocketParams()
135     sim = Simulator(rk)
136     res = sim.simulate(t_coast=rk.t_coast_q1, controller=None, t_shut=None)
137
138     print("=" * 72)
139     print("问题 1: 基准控制策略仿真结果")
140     print("=" * 72)

```

```

141 print(f"一子级质量流量          mdot1 = {rk.mdot1:10.2f} kg/s")
142 print(f"一子级推进剂耗尽时刻    t1    = {rk.t_burn1:10.2f} s")
143 print(f"二子级质量流量          mdot2 = {rk.mdot2:10.2f} kg/s")
144 print(f"二子级推进剂耗尽时长    tb2    = {rk.t_burn2:10.2f} s")
145 print(f"分离时一子级关机质量      = {rk.M0 - rk.mp1:10.0f} kg")
146 print(f"二子级点火质量（分离后）  = {rk.m_after_sep:10.0f} kg")
147 print("-" * 72)
148 print("各阶段边界时刻: ")
149 for name, tb in phase_boundaries(res).items():
150     print(f"   {name:<8s} t = {tb:8.2f} s" if tb is not None else f"   {name:<8s}
↪   --")
151 print("-" * 72)
152 fs = res.final_state()
153 print("二子级关机时刻状态（最终）: ")
154 print(f"   轨道高度 h          = {fs['h_km']:10.2f} km    (目标 400 km)")
155 print(f"   惯性速度 V          = {fs['v_in_mps']:10.2f} m/s  (圆轨道 {rk.v_circular
↪   :.2f} m/s)")
156 print(f"   飞行路径角 gamma     = {fs['gamma_deg']:10.4f} deg (惯性) / "
157       f"{fs['gamma_rel_deg']:10.4f} deg (相对大气)")
158 print(f"   总质量 m              = {fs['m_kg']:10.0f} kg    (结构+载荷 = {rk.ms2 + rk.
↪   m_payload:.0f} kg)")
159 res_orb = orbit_residual(res.x[-1], rk)
160 print(f"   入轨条件残差      = {res_orb}  (0 为精确入轨)")
161 print("-" * 72)
162 print("说明: 基准策略下二子级燃料耗尽即关机, 入轨条件不一定满足: ")
163 print("       超出/不足的差异即问题 2 需要修正的内容。")
164
165 # ---- 保存结果 ----
166 df = pd.DataFrame({
167     "t_s": res.t,
168     "h_km": res.h / 1e3,
169     "V_inertial_mps": res.v_in,
170     "gamma_inertial_deg": res.gamma / DEG,
171     "gamma_rel_deg": res.gamma_rel / DEG,
172     "m_kg": res.m,
173     "T_N": res.T,
174     "phase": res.phase,
175 })
176 df.to_csv(RES_DIR / "q1_full_trajectory.csv", index=False)
177
178 summary = {
179     "t1_burn1_s": rk.t_burn1,
180     "t2_ignite_s": res.t2,
181     "t_shut_s": res.t_shut,
182     "h_final_km": fs["h_km"],
183     "v_final_mps": fs["v_in_mps"],
184     "gamma_final_deg": fs["gamma_deg"],
185     "gamma_rel_final_deg": fs["gamma_rel_deg"],

```

```

186     "m_final_kg": fs["m_kg"],
187     "h_target_km": rk.H_target / 1e3,
188     "v_circular_mps": rk.v_circular,
189 }
190 pd.DataFrame([summary]).to_csv(RES_DIR / "q1_summary.csv", index=False)
191
192 plot_profiles(res, rk, "q1_flight_profiles.png")
193 plot_trajectory(res, "q1_trajectory.png")
194 print(f"已保存: {RES_DIR / 'q1_full_trajectory.csv'}, {RES_DIR / 'q1_summary."
↪  csv'}")
195 print(f"已保存: {FIG_DIR / 'q1_flight_profiles.png'}, {FIG_DIR / 'q1_trajectory"
↪  .png'}")
196
197
198 if __name__ == "__main__":
199     main()

```

C 问题二入轨打靶 q2_inscription.py

```
1 """问题 2：入轨条件反演 —— 调整滑行时间与二子级俯仰角变化率（可提前关机）。
2
3 控制策略（题目给定）：
4 - 二子级推力俯仰角以恒定速率  $k$  变化，初始俯仰角与（相对大气）速度方向一致；
5 - 二子级维持额定全推力；
6 - 关机时刻  $t_{\text{shut}}$  由入轨条件决定（可提前关机）；
7 - 目标：进入 400 km 近地圆轨道。
8
9 入轨条件（惯性系，3 个等式）：
10      $|r| = R_E + H$ ,  $|v| = \sqrt{\mu/|r|}$ ,  $r \cdot v = 0$ 
11
12 未知量 ( $t_{\text{coast}}$ ,  $k$ ,  $t_{\text{shut}}$ ) 共 3 个、方程 3 个 -> 三维打靶 (fsolve 多初值)。
13 点火前轨迹只与  $t_{\text{coast}}$  有关，缓存之；二级段单独快速积分。
14 """
15
16 from __future__ import annotations
17
18 import matplotlib
19
20 matplotlib.use("Agg")
21 import matplotlib.pyplot as plt
22 import numpy as np
23 import pandas as pd
24 from scipy.integrate import solve_ivp
25 from scipy.optimize import fsolve
26
27 from common import (
28     DEG,
29     MU,
30     RES_DIR,
31     FIG_DIR,
32     RocketParams,
33     Simulator,
34     orbit_residual,
35     rel_velocity,
36     thrust_unit,
37 )
38
39 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial"]
40 plt.rcParams["axes.unicode_minus"] = False
41
42
43 class InscriptionSolver:
44     """问题 2 三维打靶求解器。"""
45
```

```

46 def __init__(self, rk: RocketParams | None = None, rtol: float = 1e-9):
47     self.rk = rk or RocketParams()
48     self.rtol = rtol
49     self._ign_cache: dict[float, tuple[np.ndarray, float]] = {}
50
51 # -- 点火前状态缓存（只与 t_coast 有关）-----
52 def ignition_state(self, t_coast: float) -> tuple[np.ndarray, float]:
53     key = round(t_coast, 6)
54     if key not in self._ign_cache:
55         sim = Simulator(self.rk)
56         res = sim.simulate(
57             t_coast=key, controller=None,
58             t_shut=self.rk.t_burn1 + key + 1e-6,
59             rtol=self.rtol,
60         )
61         seg = res.stages[-1]
62         self._ign_cache[key] = (seg.x[0].copy(), float(seg.t[0]))
63     return self._ign_cache[key]
64
65 # -- 二级段单次积分 -----
66 def stage2_end(
67     self, x0: np.ndarray, t2: float, phi0: float,
68     k_deg_s: float, t_shut: float,
69 ) -> np.ndarray:
70     k = k_deg_s * DEG
71     rk = self.rk
72
73     def rhs(t, x):
74         p = phi0 + k * (t - t2)
75         tx_h, ty_h = thrust_unit(p, x[0], x[1])
76         T = rk.Fmax2
77         r = np.hypot(x[0], x[1])
78         ax = -MU / r**3 * x[0] + T / x[4] * tx_h
79         ay = -MU / r**3 * x[1] + T / x[4] * ty_h
80         return np.array([x[2], x[3], ax, ay, -T / (rk.Isp2 * 9.80665)])
81
82     sol = solve_ivp(
83         rhs, (t2, t_shut), x0, method="DOP853",
84         rtol=self.rtol, atol=[1e-2, 1e-2, 1e-4, 1e-4, 1e-2],
85     )
86     return sol.y[:, -1]
87
88 # -- 残差 -----
89 def residual(self, z: np.ndarray) -> np.ndarray:
90     t_coast, k_deg_s, t_shut = z
91     x0, t2 = self.ignition_state(t_coast)
92     vr_x, vr_y = rel_velocity(x0[0], x0[1], x0[2], x0[3])
93     phi0 = np.arctan2(vr_x * x0[0] + vr_y * x0[1], vr_x * (-x0[1]) + vr_y * x0[0])

```

```

94         if t_shut > t2 + self.rk.t_burn2:    # 超出燃尽上限 -> 罚
95             return np.array([1.0, 1.0, 0.0])
96         xf = self.stage2_end(x0, t2, phi0, k_deg_s, t_shut)
97         return orbit_residual(xf, self.rk)
98
99     # -- 多初值打靶 -----
100     def solve(self, starts=None) -> list[tuple[np.ndarray, float]]:
101         if starts is None:
102             starts = []
103             for tc in [0.0, 30.0, 60.0, 90.0, 120.0, 150.0, 180.0]:
104                 for k in [-0.4, -0.2, -0.1, -0.05, 0.0, 0.1, 0.2]:
105                     for ts in [350.0, 450.0, 550.0, 650.0]:
106                         starts.append((tc, k, ts))
107
108         sols = {}
109         for z0 in starts:
110             # 可行域检查: 燃尽上限保护
111             x0, t2 = self.ignition_state(z0[0])
112             if z0[2] > t2 + self.rk.t_burn2:
113                 continue
114             try:
115                 sol, info, ier, msg = fsolve(
116                     self.residual, np.array(z0, float), full_output=True, xtol=1e
117                     ↪ -11,
118                     )
119                 if ier == 1:
120                     n = float(np.linalg.norm(self.residual(sol)))
121                     key = (round(sol[0], 1), round(sol[1], 4))
122                     if key not in sols or n < sols[key][1]:
123                         sols[key] = (sol, n)
124             except Exception:
125                 pass
126
127         return sorted(sols.values(), key=lambda kv: kv[1])
128
129 def main() -> None:
130     rk = RocketParams()
131     solver = InscriptionSolver(rk)
132
133     print("=" * 72)
134     print("问题 2: 入轨打靶 (网格多初值 + fsolve)")
135     print("=" * 72)
136     sols = solver.solve()
137     print(f"找到 {len(sols)} 组收敛根 (按残差排序): ")
138     for i, (sol, n) in enumerate(sols[:6]):
139         print(f"    #{i+1}: t_c={sol[0]:8.3f} s    k={sol[1]:8.4f} deg/s    "
140             f"t_shut={sol[2]:8.3f} s    |res|={n:.2e}")
141
142     if not sols:

```

```

141     print("未找到可行解！")
142     return
143
144 sol, n = sols[0]
145 t_coast, k_deg_s, t_shut = sol
146
147 # ---- 完整重仿验证（用 Simulator 的完整流程，含事件检测与燃尽保护） ----
148 x0, t2 = solver.ignition_state(t_coast)
149 vr_x, vr_y = rel_velocity(x0[0], x0[1], x0[2], x0[3])
150 phi0 = np.arctan2(vr_x * x0[0] + vr_y * x0[1], vr_x * (-x0[1]) + vr_y * x0[0])
151 k = k_deg_s * DEG
152
153 def controller(t):
154     return phi0 + k * (t - t2)
155
156 sim = Simulator(rk)
157 res = sim.simulate(t_coast=t_coast, controller=controller, t_shut=t_shut)
158 fs_ = res.final_state()
159
160 print("-" * 72)
161 print("最终验证（入轨时刻状态）：")
162 print(f"  滑行时间      t_c      = {t_coast:.3f} s")
163 print(f"  俯仰角速率    k        = {k_deg_s:.4f} deg/s")
164 print(f"  初始俯仰角    phi0      = {phi0 / DEG:.4f} deg（与点火时刻速度方向一致）")
165 print(f"  一子级关机    t1        = {res.t1:.3f} s；二子级点火 t2 = {t2:.3f} s")
166 print(f"  关机时刻      t_shut    = {t_shut:.3f} s（燃尽上限 {t2 + rk.t_burn2:.3f} s  

↳ ）")
167 print(f"  轨道高度      h        = {fs_['h_km']:.4f} km（目标 400）")
168 print(f"  惯性速度      V        = {fs_['v_in_mps']:.3f} m/s（目标 {rk.v_circular  

↳ :.3f}）")
169 print(f"  飞行路径角    gamma     = {fs_['gamma_deg']:.5f} deg（目标 0）")
170 print(f"  关机质量      m        = {fs_['m_kg']:.1f} kg（剩余推进剂 "
171       f"{fs_['m_kg'] - rk.ms2 - rk.m_payload:.1f} kg）")
172 print(f"  入轨残差      = {orbit_residual(res.x[-1], rk)}")
173
174 # ---- 保存 & 绘图 ----
175 df = pd.DataFrame({
176     "t_s": res.t, "h_km": res.h / 1e3, "V_mps": res.v_in,
177     "gamma_deg": res.gamma / DEG, "m_kg": res.m, "phase": res.phase,
178 })
179 df.to_csv(RES_DIR / "q2_trajectory.csv", index=False)
180 pd.DataFrame([
181     "t_coast_s": t_coast, "k_deg_s": k_deg_s, "t_shut_s": t_shut,
182     "phi0_deg": phi0 / DEG,
183     "h_final_km": fs_["h_km"], "v_final_mps": fs_["v_in_mps"],
184     "gamma_final_deg": fs_["gamma_deg"],
185     "m_final_kg": fs_["m_kg"],
186     "propellant_remaining_kg": fs_["m_kg"] - rk.ms2 - rk.m_payload,

```



```

187     "residual_norm": n,
188 ]]).to_csv(RES_DIR / "q2_summary.csv", index=False)
189
190 fig, axes = plt.subplots(3, 1, figsize=(10, 9))
191 ax = axes[0]
192 ax.plot(res.t / 60, res.h / 1e3, lw=1.6, color="tab:blue")
193 ax.axhline(400, ls="--", color="gray", label="目标轨道 400 km")
194 ax.axvline(res.t2 / 60, ls=":", color="k", alpha=0.4, label="二子级点火")
195 ax.axvline(res.t_shut / 60, ls=":", color="r", alpha=0.5, label="关机")
196 ax.set_xlabel("时间 t (min)"); ax.set_ylabel("高度 h (km)")
197 ax.set_title("问题 2 入轨轨迹: h(t)"); ax.legend(fontsize=8); ax.grid(alpha
↵ =0.4)
198
199 ax = axes[1]
200 ax.plot(res.t / 60, res.v_in / 1e3, lw=1.6, color="tab:red")
201 ax.axhline(rk.v_circular / 1e3, ls="--", color="gray",
202            label=f"圆轨道速度 {rk.v_circular/1e3:.2f} km/s")
203 ax.set_xlabel("时间 t (min)"); ax.set_ylabel("惯性速度 V (km/s)")
204 ax.set_title("V(t)"); ax.legend(fontsize=8); ax.grid(alpha=0.4)
205
206 ax = axes[2]
207 ax.plot(res.t / 60, res.gamma / DEG, lw=1.6, color="tab:green")
208 ax.axhline(0, ls="--", color="gray", label="gamma = 0")
209 ax.set_xlabel("时间 t (min)"); ax.set_ylabel("飞行路径角 gamma (deg)")
210 ax.set_title("gamma(t)"); ax.legend(fontsize=8); ax.grid(alpha=0.4)
211 fig.tight_layout()
212 fig.savefig(FIG_DIR / "q2_orbit_inscription.png", dpi=150)
213 plt.close(fig)
214 print(f"已保存: {RES_DIR / 'q2_trajectory.csv'}, {RES_DIR / 'q2_summary.csv'},
↵ "
215       f"{FIG_DIR / 'q2_orbit_inscription.png'}")
216
217
218 if __name__ == "__main__":
219     main()

```

D 问题三燃料最省 q3_fuel_opt.py

```
1 """问题 3：设计滑行时间与二子级俯仰角优化控制策略，使二子级燃料消耗最省。
2
3 优化问题：
4     min J = 二子级消耗推进剂 = m(t2+) - m(t_shut)    (等价 max m(t_shut))
5     s.t. 入轨条件 3 等式 (|r|=a, |v|=sqrt(mu/a), r·v=0)
6         俯仰角 phi(t) 以分段线性控制律参数化 (N 段节点值)
7         关机时刻 t_shut 自由 (由入轨条件决定，可提前关机)
8
9 求解：控制参数化 (分段线性) + 直接打靶 + SLSQP (带入轨等式约束)，
10      多初值 (含问题 2 的解作为初值) 避免局部解。
11 """
12
13 from __future__ import annotations
14
15 import matplotlib
16
17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19 import numpy as np
20 import pandas as pd
21 from scipy.integrate import solve_ivp
22 from scipy.optimize import minimize
23
24 from common import (
25     DEG,
26     MU,
27     RES_DIR,
28     FIG_DIR,
29     RocketParams,
30     Simulator,
31     orbit_residual,
32     rel_velocity,
33     thrust_unit,
34 )
35
36 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial"]
37 plt.rcParams["axes.unicode_minus"] = False
38
39 N_PHI = 5          # 俯仰角控制节点数 (分段线性)
40 N_GRID_TIMES = 6   # 初值扫描点火前网格数量
41
42
43 class FuelOptimizer:
44     """问题 3：燃料最省优化器 (控制参数化 + SLSQP) 。"""
45
```

```

46 def __init__(self, rk: RocketParams | None = None, n_phi: int = N_PHI, rtol:
↪ float = 1e-9):
47     self.rk = rk or RocketParams()
48     self.n_phi = n_phi
49     self.rtol = rtol
50     self._ign_cache: dict[float, tuple[np.ndarray, float]] = {}
51
52 def ignition_state(self, t_coast: float) -> tuple[np.ndarray, float]:
53     key = round(t_coast, 6)
54     if key not in self._ign_cache:
55         sim = Simulator(self.rk)
56         res = sim.simulate(
57             t_coast=key, controller=None,
58             t_shut=self.rk.t_burn1 + key + 1e-6, rtol=self.rtol,
59         )
60         seg = res.stages[-1]
61         self._ign_cache[key] = (seg.x[0].copy(), float(seg.t[0]))
62     return self._ign_cache[key]
63
64 def phi0_of(self, x0: np.ndarray) -> float:
65     """点火时刻相对速度路径角 = 初始俯仰角基准。"""
66     vr_x, vr_y = rel_velocity(x0[0], x0[1], x0[2], x0[3])
67     return float(np.arctan2(vr_x * x0[0] + vr_y * x0[1], vr_x * (-x0[1]) + vr_y *
↪ x0[0]))
68
69 # -- 二级段积分 (phi 用控制节点线性插值, 节点时间线性分布于 [t2, t_shut]) --
70 def stage2_end(self, x0, t2, phi0, phi_nodes, t_shut):
71     rk = self.rk
72     n = self.n_phi
73     # 节点时刻 (相对 t2), 等距分布于 [0, t_shut-t2]
74     tau_nodes = np.linspace(0.0, 1.0, n)
75     # phi 节点值与 phi0 拼接: 第一段起点即 phi0 -> 用 (phi0, phi_nodes) 长度 n
↪ +1?
76     # 更简单: 节点值为绝对俯仰角 [deg], 首点固定 phi0, 其余优化
77     phi_abs = np.concatenate(([phi0 / DEG], phi_nodes)) # 长度 n
78
79     def phi_at(tau):
80         return np.interp(tau, tau_nodes, phi_abs) * DEG
81
82     def rhs(t, x):
83         tau = (t - t2) / (t_shut - t2)
84         p = phi_at(tau)
85         tx_h, ty_h = thrust_unit(p, x[0], x[1])
86         T = rk.Fmax2
87         r = np.hypot(x[0], x[1])
88         ax = -MU / r**3 * x[0] + T / x[4] * tx_h
89         ay = -MU / r**3 * x[1] + T / x[4] * ty_h
90         return np.array([x[2], x[3], ax, ay, -T / (rk.Isp2 * 9.80665)])

```

```

91     sol = solve_ivp(rhs, (t2, max(t_shut, t2 + 1e-3)), x0, method="DOP853",
92                     rtol=self.rtol, atol=[1e-2, 1e-2, 1e-4, 1e-4, 1e-2])
93     return sol.y[:, -1]
94
95 # -- 目标 + 约束 -----
96 def evaluate(self, z: np.ndarray):
97     """决策向量 z = [t_coast, phi_1..phi_{n-1}, t_shut]。返回 (目标, 残差)。"""
98     t_coast = z[0]
99     phi_nodes = z[1:-1]
100     t_shut = z[-1]
101     x0, t2 = self.ignition_state(t_coast)
102     phi0 = self.phi0_of(x0)
103     m_ignit = x0[4]
104     if t_shut < t2 + 1.0 or t_shut > t2 + self.rk.t_burn2:
105         # 罚: 理想目标给大值
106         return 1e6 + abs(t_shut - t2), np.array([1.0, 1.0, 1.0])
107     xf = self.stage2_end(x0, t2, phi0, phi_nodes, t_shut)
108     prop_used = m_ignit - xf[4]
109     res = orbit_residual(xf, self.rk)
110     return prop_used, res
111
112 def objective(self, z: np.ndarray) -> float:
113     prop_used, res = self.evaluate(z)
114     # 罚函数: 入轨残差违反时惩罚 (SLSQP 也用约束, 双保险)
115     penalty = 1e3 * float(np.sum(res**2))
116     return prop_used + penalty
117
118 def constraint_eq(self, z: np.ndarray) -> np.ndarray:
119     _, res = self.evaluate(z)
120     return res
121
122 # -- 求解 -----
123 def _solve_one(self, z0: np.ndarray, maxiter: int) -> tuple | None:
124     """单初值 SLSQP (供并行调用)。"""
125     cons = {"type": "eq", "fun": lambda z: self.constraint_eq(z) * 1e2}
126     opt = minimize(
127         self.objective, z0, method="SLSQP",
128         bounds=[(0.0, 400.0)] + [(-90.0, 90.0)] * (self.n_phi - 1)
129             + [(0.0, 1e3)],
130         constraints=[cons],
131         options={"ftol": 1e-8, "maxiter": maxiter, "disp": False},
132     )
133     prop_used, res = self.evaluate(opt.x)
134     if np.linalg.norm(res) < 1e-5:
135         return opt.x, prop_used, float(np.linalg.norm(res))
136     return None
137
138

```

```

139 def solve(self, starts: list[np.ndarray], maxiter: int = 60, workers: int = 8):
140     from concurrent.futures import ProcessPoolExecutor
141     results = []
142     with ProcessPoolExecutor(max_workers=workers) as pool:
143         futures = [pool.submit(self._solve_one, z0, maxiter) for z0 in starts]
144         for fut in futures:
145             r = fut.result()
146             if r is not None:
147                 results.append(r)
148         results.sort(key=lambda r: r[1])
149     return results
150
151
152 def make_starts(rk, n_phi) -> list[np.ndarray]:
153     """初值集合：覆盖滑行时间 0~300s、俯仰角律多种形状（线性/凸/凹）。"""
154     s = Simulator(rk)
155     starts = []
156     bt = rk.t_burn2
157     opt0 = FuelOptimizer(rk)
158     for tc in [0.0, 60.0, 90.0, 113.0, 130.0, 160.0, 200.0, 250.0, 300.0]:
159         res = s.simulate(t_coast=tc, controller=None, t_shut=rk.t_burn1 + tc + 1e
160 ↪ -6)
161         x0 = res.stages[-1].x[0]
162         phi0 = opt0.phi0_of(x0)
163         t2 = float(res.stages[-1].t[0])
164         for shape in [-1.0, -0.3, 0.0, 0.3, 1.0]:
165             z = np.zeros(n_phi + 1)
166             z[0] = tc
167             # phi(tau) = phi0 + shape * (phi_land - phi0) * tau^p, p=1 线性、p>1
168 ↪ 凸、p<1 凹
169             # 取终端俯仰角目标 phi_land 为 5deg 附近（gamma 需归零），用形状参数生
170 ↪ 成
171             phi_land = 4.0
172             for j in range(n_phi - 1):
173                 tau = (j + 1) / n_phi
174                 p = 1.0 if shape == 0 else (2.0 if shape > 0 else 0.5)
175                 z[1 + j] = phi0 / DEG + (phi_land - phi0 / DEG) * tau**p
176                 z[-1] = t2 + 0.85 * bt
177             starts.append(z)
178     return starts
179
180
181 def main() -> None:
182     rk = RocketParams()
183     opt = FuelOptimizer(rk)
184     print("=" * 72)
185     print("问题 3：二子级燃料最省（滑行时间 + 俯仰角分段线性控制）")
186     print(f"控制节点数 N_phi = {opt.n_phi}（首点固定为初始俯仰角）")

```

```

184     print("=" * 72)
185
186     starts = make_starts(rk, opt.n_phi)
187     print(f"初值数量: {len(starts)}")
188     results = opt.solve(starts)
189     print(f"收敛解数量: {len(results)}")
190     if not results:
191         print("未找到满足入轨条件的解!")
192         return
193
194     z_best, prop_used, nres = results[0]
195     # 双精度重算并输出
196     z_best = np.array(z_best, float)
197     prop_used, res = opt.evaluate(z_best)
198     t_coast, phi_nodes, t_shut = z_best[0], z_best[1:-1], z_best[-1]
199     x0, t2 = opt.ignition_state(t_coast)
200     phi0 = opt.phi0_of(x0)
201
202     print("-" * 72)
203     print(f"最优解: ")
204     print(f"   滑行时间      t_c      = {t_coast:.4f} s")
205     print(f"   关机时刻      t_shut = {t_shut:.4f} s (点火 t2 = {t2:.4f} s, "
206           f"燃尽上限 {t2 + rk.t_burn2:.4f} s) ")
207     print(f"   俯仰角节点 (deg): ", end="")
208     tau_nodes = np.linspace(0.0, 1.0, opt.n_phi)
209     for k, v in zip(tau_nodes, np.concatenate(([phi0 / DEG], phi_nodes))):
210         print(f" [{k:.2f}]:{v:.3f}", end="")
211     print()
212     print(f"   二子级消耗推进剂 = {prop_used:.1f} kg")
213     print(f"   关机质量          = {x0[4] - prop_used:.1f} kg (剩余推进剂 "
214           f"{x0[4] - prop_used - rk.ms2 - rk.m_payload:.1f} kg) ")
215     print(f"   入轨残差 |res|    = {nres:.2e}")
216
217     # 与问题 2 解对比
218     q2 = pd.read_csv(RES_DIR / "q2_summary.csv").iloc[0]
219     print("-" * 72)
220     print(f"对比问题 2 (恒定俯仰角速率 k = {q2['k_deg_s']:.4f} deg/s): ")
221     print(f"   Q2 消耗推进剂 = {rk.mp2 - q2['propellant_remaining_kg']:.1f} kg")
222     print(f"   Q3 消耗推进剂 = {prop_used:.1f} kg (节省 {rk.mp2 - q2['"
223     ↪ propellant_remaining_kg'] - prop_used:.1f} kg) ")
224
225     # ---- 完整仿真重新验证 ----
226     def controller(t):
227         tau = (t - t2) / (t_shut - t2)
228         phi_abs = np.concatenate(([phi0 / DEG], phi_nodes))
229         return float(np.interp(tau, tau_nodes, phi_abs) * DEG)
230
231     sim = Simulator(rk)

```

```

231 res = sim.simulate(t_coast=t_coast, controller=controller, t_shut=t_shut)
232 fs_ = res.final_state()
233 import numpy as _np
234 x0v, t2v = opt.ignition_state(t_coast)
235 print(f"[dbg] phi0_match={abs(phi0 - opt.phi0_of(x0v)):.2e} t2v={t2v:.6f} "
236       f"t_shut={t_shut:.6f} t2={t2:.6f}")
237 print(f"[dbg] evaluate res={opt.evaluate(z_best)[1]} sim res={orbit_residual(
↵ res.x[-1], rk)}")
238 print("-" * 72)
239 print("完整仿真验证: ")
240 print(f" h = {fs_['h_km']:.4f} km, V = {fs_['v_in_mps']:.3f} m/s, "
241       f"gamma = {fs_['gamma_deg']:.5f} deg")
242 print(f" 入轨残差 = {orbit_residual(res.x[-1], rk)}")
243
244 # ---- 保存 ----
245 pd.DataFrame([
246     "t_coast_s": t_coast, "t_shut_s": t_shut,
247     "phi_nodes_deg": " | ".join(f"{v:.4f}" for v in phi_nodes),
248     "phi0_deg": phi0 / DEG,
249     "propellant_used_kg": prop_used,
250     "m_final_kg": fs_["m_kg"],
251     "propellant_remaining_kg": fs_["m_kg"] - rk.ms2 - rk.m_payload,
252     "residual_norm": nres,
253 ]).to_csv(RES_DIR / "q3_summary.csv", index=False)
254 df = pd.DataFrame({
255     "t_s": res.t, "h_km": res.h / 1e3, "V_mps": res.v_in,
256     "gamma_deg": res.gamma / DEG, "m_kg": res.m,
257     "phi_deg": res.phi / DEG, "phase": res.phase,
258 })
259 df.to_csv(RES_DIR / "q3_trajectory.csv", index=False)
260
261 # ---- 绘图: 控制律与轨迹 ----
262 fig, axes = plt.subplots(2, 2, figsize=(12, 8))
263 ax = axes[0, 0]
264 ax.plot(res.t / 60, res.h / 1e3, lw=1.6)
265 ax.axhline(400, ls="--", color="gray", label="目标轨道")
266 ax.set_xlabel("t (min)"); ax.set_ylabel("h (km)"); ax.legend(fontsize=8)
267 ax.set_title("高度"); ax.grid(alpha=0.4)
268 ax = axes[0, 1]
269 ax.plot(res.t / 60, res.v_in / 1e3, lw=1.6, color="tab:red")
270 ax.axhline(rk.v_circular / 1e3, ls="--", color="gray", label="圆轨道速度")
271 ax.set_xlabel("t (min)"); ax.set_ylabel("V (km/s)"); ax.legend(fontsize=8)
272 ax.set_title("惯性速度"); ax.grid(alpha=0.4)
273 ax = axes[1, 0]
274 ax.plot(res.t / 60, res.gamma / DEG, lw=1.6, color="tab:green")
275 ax.axhline(0, ls="--", color="gray")
276 ax.set_xlabel("t (min)"); ax.set_ylabel("gamma (deg)")
277 ax.set_title("飞行路径角"); ax.grid(alpha=0.4)

```

```

278     ax = axes[1, 1]
279     ax.plot(res.t[res.t >= t2] / 60, res.phi[res.t >= t2] / DEG, lw=1.6, color="tab
↵ :purple")
280     ax.set_xlabel("t (min)"); ax.set_ylabel("俯仰角 phi (deg)")
281     ax.set_title("二子级俯仰角控制律 (分段线性)"); ax.grid(alpha=0.4)
282     fig.suptitle("问题 3: 燃料最省最优轨迹与最优控制", fontsize=13)
283     fig.tight_layout(rect=(0, 0, 1, 0.96))
284     fig.savefig(FIG_DIR / "q3_fuel_optimal.png", dpi=150)
285     plt.close(fig)
286     print(f"已保存: {RES_DIR / 'q3_summary.csv'}, {RES_DIR / 'q3_trajectory.csv'},
↵ "
287           f"{FIG_DIR / 'q3_fuel_optimal.png'}")
288
289
290 if __name__ == "__main__":
291     main()

```


E 问题四节流优化 q4_throttle_opt.py

```
1 """问题 4：二子级发动机具备推力节流能力（60%~100% 额定推力）下，
2 重新设计滑行时间与二子级推力大小、俯仰角的优化控制策略，使燃料消耗最省。
3
4 优化问题：
5     min J = 二子级消耗推进剂 = integral mdotdt = integral (sigma(t) * Fmax2) / (
6         ↪ Isp2 * g0) dt
7     s.t. 入轨条件 3 等式 ( $|r|=a$ ,  $|v|=\sqrt{\mu/a}$ ,  $r \cdot v=0$ )
8         节流比 sigma(t) in [0.6, 1] (分段线性参数化)
9         俯仰角 phi(t) 分段线性参数化 (首点取点火时刻速度方向)
10        关机时刻 t_shut 自由 (由入轨条件决定)
11
12 求解：控制参数化 (phi 与 sigma 均 N 节点分段线性) + 直接打靶 + SLSQP 多初值。
13
14 from __future__ import annotations
15
16 import matplotlib
17
18 matplotlib.use("Agg")
19 import matplotlib.pyplot as plt
20 import numpy as np
21 import pandas as pd
22 from scipy.integrate import solve_ivp
23 from scipy.optimize import minimize
24
25 from common import (
26     DEG,
27     MU,
28     RES_DIR,
29     FIG_DIR,
30     RocketParams,
31     Simulator,
32     orbit_residual,
33     rel_velocity,
34     thrust_unit,
35 )
36
37 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial"]
38 plt.rcParams["axes.unicode_minus"] = False
39
40 N_SIGMA = 4 # 节流比控制节点数
41 SIGMA_MIN = 0.6
42
43
44 class ThrottleOptimizer:
```

```

45 """问题 4：节流 + 俯仰角联合优化（控制参数化 + SLSQP）。"""
46
47 def __init__(self, rk: RocketParams | None = None,
48               n_phi: int = 5, n_sigma: int = N_SIGMA, rtol: float = 1e-9):
49     self.rk = rk or RocketParams()
50     self.n_phi = n_phi
51     self.n_sigma = n_sigma
52     self.rtol = rtol
53     self._ign_cache: dict[float, tuple[np.ndarray, float]] = {}
54
55 def ignition_state(self, t_coast: float) -> tuple[np.ndarray, float]:
56     key = round(t_coast, 6)
57     if key not in self._ign_cache:
58         sim = Simulator(self.rk)
59         res = sim.simulate(
60             t_coast=key, controller=None,
61             t_shut=self.rk.t_burn1 + key + 1e-6, rtol=self.rtol,
62         )
63         seg = res.stages[-1]
64         self._ign_cache[key] = (seg.x[0].copy(), float(seg.t[0]))
65     return self._ign_cache[key]
66
67 def phi0_of(self, x0: np.ndarray) -> float:
68     vrx, vry = rel_velocity(x0[0], x0[1], x0[2], x0[3])
69     return float(np.arctan2(vrx * x0[0] + vry * x0[1], vrx * (-x0[1]) + vry *
    ↪ x0[0]))
70
71 # -- 二级段积分 -----
72 def stage2_end(self, x0, t2, phi0, phi_nodes, sigma_nodes, t_shut):
73     rk = self.rk
74     n_phi, n_sig = self.n_phi, self.n_sigma
75     tau_phi = np.linspace(0.0, 1.0, n_phi)
76     tau_sig = np.linspace(0.0, 1.0, n_sig)
77     phi_abs = np.concatenate(([phi0 / DEG], phi_nodes)) # 长度 n_phi
78     sig_arr = np.asarray(sigma_nodes, float) # 长度 n_sig
79
80     def interp(tau, tau_nodes, vals):
81         return float(np.interp(tau, tau_nodes, vals))
82
83     def rhs(t, x):
84         tau = (t - t2) / (t_shut - t2)
85         p = interp(tau, tau_phi, phi_abs) * DEG
86         sig = float(np.clip(interp(tau, tau_sig, sig_arr), SIGMA_MIN, 1.0))
87         tx_h, ty_h = thrust_unit(p, x[0], x[1])
88         T = sig * rk.Fmax2
89         r = np.hypot(x[0], x[1])
90         ax = -MU / r**3 * x[0] + T / x[4] * tx_h
91         ay = -MU / r**3 * x[1] + T / x[4] * ty_h

```

```

92         mdot = -T / (rk.Isp2 * 9.80665)
93         return np.array([x[2], x[3], ax, ay, mdot])
94
95     sol = solve_ivp(rhs, (t2, max(t_shut, t2 + 1e-3)), x0, method="DOP853",
96                     rtol=self.rtol, atol=[1e-2, 1e-2, 1e-4, 1e-4, 1e-2])
97     return sol.y[:, -1]
98
99     # -- 目标 + 约束 -----
100     def evaluate(self, z: np.ndarray):
101         """z = [t_coast, phi_1..phi_{n_phi-1}, sigma_1..sigma_{n_sigma}, t_shut]"""
102         t_coast = z[0]
103         phi_nodes = z[1:1 + (self.n_phi - 1)]
104         sigma_nodes = z[1 + (self.n_phi - 1): 1 + (self.n_phi - 1) + self.n_sigma]
105         t_shut = z[-1]
106         x0, t2 = self.ignition_state(t_coast)
107         phi0 = self.phi0_of(x0)
108         if t_shut < t2 + 1.0 or t_shut > t2 + self.rk.t_burn2:
109             return 1e6 + abs(t_shut - t2), np.array([1.0, 1.0, 1.0])
110         xf = self.stage2_end(x0, t2, phi0, phi_nodes, sigma_nodes, t_shut)
111         prop_used = x0[4] - xf[4]
112         res = orbit_residual(xf, self.rk)
113         return prop_used, res
114
115     def objective(self, z: np.ndarray) -> float:
116         prop_used, res = self.evaluate(z)
117         penalty = 1e3 * float(np.sum(res**2))
118         return prop_used + penalty
119
120     def constraint_eq(self, z: np.ndarray) -> np.ndarray:
121         _, res = self.evaluate(z)
122         return res
123
124     def _solve_one(self, z0: np.ndarray, maxiter: int, bounds) -> tuple | None:
125         cons = {"type": "eq", "fun": lambda z: self.constraint_eq(z) * 1e2}
126         opt = minimize(
127             self.objective, z0, method="SLSQP", bounds=bounds,
128             constraints=cons,
129             options={"ftol": 1e-8, "maxiter": maxiter, "disp": False},
130         )
131         prop_used, res = self.evaluate(opt.x)
132         if np.linalg.norm(res) < 1e-5:
133             return opt.x, prop_used, float(np.linalg.norm(res))
134         return None
135
136     def solve(self, starts: list[np.ndarray], maxiter: int = 80, workers: int = 8):
137         from concurrent.futures import ProcessPoolExecutor
138         n_phi, n_sig = self.n_phi, self.n_sigma
139         bounds = [(0.0, 400.0)] + [(-90.0, 90.0)] * (n_phi - 1) \

```

```

140         + [(SIGMA_MIN, 1.0)] * n_sig + [(0.0, 1e3)]
141     results = []
142     with ProcessPoolExecutor(max_workers=workers) as pool:
143         futures = [pool.submit(self._solve_one, z0, maxiter, bounds) for z0 in
↪ starts]
144         for fut in futures:
145             r = fut.result()
146             if r is not None:
147                 results.append(r)
148     results.sort(key=lambda r: r[1])
149     return results
150
151
152 def make_starts_q4(rk, n_phi, n_sigma) -> list[np.ndarray]:
153     """初值: Q3 最优解 (全推力) + 节流 0.6/0.8 + 多滑行时间组合。"""
154     s = Simulator(rk)
155     starts = []
156     opt0 = ThrottleOptimizer(rk, n_phi, n_sigma)
157     # 1) 从 Q3 解构造 (若存在): t_c=90, 线性俯仰律, sigma=1
158     try:
159         q3 = pd.read_csv(RES_DIR / "q3_summary.csv").iloc[0]
160         tc_q3 = float(q3["t_coast_s"])
161         ts_q3 = float(q3["t_shut_s"])
162         phi0_q3 = float(q3["phi0_deg"])
163         phi_nodes_q3 = [float(v) for v in str(q3["phi_nodes_deg"]).split(" | ")]
164         for sig_level in [0.6, 0.8, 0.9, 1.0]:
165             z = np.zeros(1 + (n_phi - 1) + n_sigma + 1)
166             z[0] = tc_q3
167             z[1:1 + (n_phi - 1)] = phi_nodes_q3[:n_phi - 1]
168             z[1 + (n_phi - 1): 1 + (n_phi - 1) + n_sigma] = sig_level
169             z[-1] = ts_q3 * (0.95 if sig_level < 1.0 else 1.0)
170             starts.append(z)
171     except Exception:
172         pass
173     # 2) 广泛网格
174     for tc in [30.0, 60.0, 90.0, 113.0, 150.0, 200.0, 260.0]:
175         res = s.simulate(t_coast=tc, controller=None, t_shut=rk.t_burn1 + tc + 1e
↪ -6)
176         x0 = res.stages[-1].x[0]
177         phi0 = opt0.phi0_of(x0)
178         t2 = float(res.stages[-1].t[0])
179         for sig_level in [0.6, 0.8, 1.0]:
180             for k0 in [-0.2, -0.1, -0.05, 0.0]:
181                 z = np.zeros(1 + (n_phi - 1) + n_sigma + 1)
182                 z[0] = tc
183                 phi_land = 4.0
184                 for j in range(n_phi - 1):
185                     tau = (j + 1) / n_phi

```

```

186         z[1 + j] = phi0 / DEG + (phi_land - phi0 / DEG) * tau
187     for j in range(n_sigma):
188         z[1 + (n_phi - 1) + j] = sig_level
189     z[-1] = t2 + 0.85 * rk.t_burn2
190     starts.append(z)
191     return starts
192
193
194 def main() -> None:
195     rk = RocketParams()
196     opt = ThrottleOptimizer(rk)
197     print("=" * 72)
198     print("问题 4：推力节流（60%~100%）下二子级燃料最省")
199     print(f"控制：俯仰角 {opt.n_phi} 节点分段线性 + 节流比 {opt.n_sigma} 节点分段线
    ↪ 性")
200     print("=" * 72)
201
202     starts = make_starts_q4(rk, opt.n_phi, opt.n_sigma)
203     print(f"初值数量：{len(starts)}")
204     results = opt.solve(starts)
205     print(f"收敛解数量：{len(results)}")
206     if not results:
207         print("未找到满足入轨条件的解！")
208         return
209
210     z_best, prop_used, nres = results[0]
211     prop_used, res = opt.evaluate(np.array(z_best, float))
212     z_best = np.array(z_best)
213     t_coast = z_best[0]
214     phi_nodes = z_best[1:1 + (opt.n_phi - 1)]
215     sigma_nodes = z_best[1 + (opt.n_phi - 1): 1 + (opt.n_phi - 1) + opt.n_sigma]
216     t_shut = z_best[-1]
217     x0, t2 = opt.ignition_state(t_coast)
218     phi0 = opt.phi0_of(x0)
219
220     print("-" * 72)
221     print("最优解：")
222     print(f"    滑行时间      t_c      = {t_coast:.4f} s")
223     print(f"    关机时刻      t_shut    = {t_shut:.4f} s (点火 t2 = {t2:.4f} s, "
224           f"    燃尽上限 {t2 + rk.t_burn2:.4f} s)")
225     print(f"    俯仰角节点 (deg) : ", end="")
226     tau_phi = np.linspace(0.0, 1.0, opt.n_phi)
227     for k, v in zip(tau_phi, np.concatenate(([phi0 / DEG], phi_nodes))):
228         print(f"    [{k:.2f}]:{v:.3f}", end="")
229     print()
230     print(f"    节流比节点 : ", end="")
231     tau_sig = np.linspace(0.0, 1.0, opt.n_sigma)
232     for k, v in zip(tau_sig, sigma_nodes):

```

```

233     print(f" [{k:.2f}]:{v:.3f}", end="")
234 print()
235 print(f" 消耗推进剂   = {prop_used:.1f} kg")
236 print(f" 关机质量     = {x0[4] - prop_used:.1f} kg (剩余推进剂 "
237       f"{x0[4] - prop_used - rk.ms2 - rk.m_payload:.1f} kg)")
238 print(f" 入轨残差     = {nres:.2e}")
239
240 # 与 Q3 对比
241 try:
242     q3 = pd.read_csv(RES_DIR / "q3_summary.csv").iloc[0]
243     print("-" * 72)
244     print(f"对比问题 3 (无节流, 全推力): 消耗推进剂 {q3['propellant_used_kg']
↵ ']:.1f} kg")
245     print(f"          本问节省 {q3['propellant_used_kg'] - prop_used:.1f} kg"
↵ )
246 except Exception:
247     print(" (Q3 结果未找到, 跳过对比) ")
248
249 # ---- 完整仿真验证 ----
250 def controller(t):
251     tau = (t - t2) / (t_shut - t2)
252     phi_abs = np.concatenate(([phi0 / DEG], phi_nodes))
253     return float(np.interp(tau, tau_phi, phi_abs) * DEG)
254
255 def sigma_fn(t):
256     tau = (t - t2) / (t_shut - t2)
257     return float(np.clip(np.interp(tau, tau_sig, sigma_nodes), SIGMA_MIN, 1.0))
258
259 sim = Simulator(rk)
260 res = sim.simulate(t_coast=t_coast, controller=controller,
261                   sigma=sigma_fn, t_shut=t_shut)
262 fs_ = res.final_state()
263 print("-" * 72)
264 print("完整仿真验证: ")
265 print(f"  h = {fs_['h_km']:.4f} km, V = {fs_['v_in_mps']:.3f} m/s, "
266       f"gamma = {fs_['gamma_deg']:.5f} deg")
267 print(f" 入轨残差 = {orbit_residual(res.x[-1], rk)}")
268
269 # ---- 保存 ----
270 pd.DataFrame([
271     "t_coast_s": t_coast, "t_shut_s": t_shut,
272     "phi_nodes_deg": " | ".join(f"{v:.4f}" for v in phi_nodes),
273     "sigma_nodes": " | ".join(f"{v:.4f}" for v in sigma_nodes),
274     "phi0_deg": phi0 / DEG,
275     "propellant_used_kg": prop_used,
276     "m_final_kg": fs_["m_kg"],
277     "propellant_remaining_kg": fs_["m_kg"] - rk.ms2 - rk.m_payload,
278     "residual_norm": nres,

```

```

279     })).to_csv(RES_DIR / "q4_summary.csv", index=False)
280     df = pd.DataFrame({
281         "t_s": res.t, "h_km": res.h / 1e3, "V_mps": res.v_in,
282         "gamma_deg": res.gamma / DEG, "m_kg": res.m,
283         "phi_deg": res.phi / DEG, "sigma": res.sigma, "phase": res.phase,
284     })
285     df.to_csv(RES_DIR / "q4_trajectory.csv", index=False)
286
287     # ---- 绘图 ----
288     fig, axes = plt.subplots(2, 3, figsize=(14, 8))
289     ax = axes[0, 0]
290     ax.plot(res.t / 60, res.h / 1e3, lw=1.6)
291     ax.axhline(400, ls="--", color="gray")
292     ax.set_xlabel("t (min)"); ax.set_ylabel("h (km)")
293     ax.set_title("高度"); ax.grid(alpha=0.4)
294     ax = axes[0, 1]
295     ax.plot(res.t / 60, res.v_in / 1e3, lw=1.6, color="tab:red")
296     ax.axhline(rk.v_circular / 1e3, ls="--", color="gray")
297     ax.set_xlabel("t (min)"); ax.set_ylabel("V (km/s)")
298     ax.set_title("惯性速度"); ax.grid(alpha=0.4)
299     ax = axes[0, 2]
300     ax.plot(res.t / 60, res.gamma / DEG, lw=1.6, color="tab:green")
301     ax.axhline(0, ls="--", color="gray")
302     ax.set_xlabel("t (min)"); ax.set_ylabel("gamma (deg)")
303     ax.set_title("飞行路径角"); ax.grid(alpha=0.4)
304     ax = axes[1, 0]
305     mask = res.t >= t2
306     ax.plot(res.t[mask] / 60, res.phi[mask] / DEG, lw=1.6, color="tab:purple")
307     ax.set_xlabel("t (min)"); ax.set_ylabel("俯仰角 phi (deg)")
308     ax.set_title("二子级俯仰角控制律"); ax.grid(alpha=0.4)
309     ax = axes[1, 1]
310     ax.plot(res.t[mask] / 60, res.sigma[mask], lw=1.6, color="tab:orange")
311     ax.axhline(0.6, ls="--", color="gray", label="0.6 下限")
312     ax.axhline(1.0, ls="--", color="gray", label="1.0 上限")
313     ax.set_xlabel("t (min)"); ax.set_ylabel("节流比 sigma")
314     ax.set_title("节流比控制律"); ax.legend(fontsize=8); ax.grid(alpha=0.4)
315     ax = axes[1, 2]
316     ax.plot(res.t[mask] / 60, res.m[mask] / 1e3, lw=1.6, color="tab:brown")
317     ax.set_xlabel("t (min)"); ax.set_ylabel("m (t)")
318     ax.set_title("二子级质量"); ax.grid(alpha=0.4)
319     fig.suptitle("问题 4: 推力节流下燃料最省最优解", fontsize=13)
320     fig.tight_layout(rect=(0, 0, 1, 0.96))
321     fig.savefig(FIG_DIR / "q4_throttle_optimal.png", dpi=150)
322     plt.close(fig)
323     print(f"已保存: {RES_DIR / 'q4_summary.csv'}, {RES_DIR / 'q4_trajectory.csv'},
324           ↪ "
325           f"{FIG_DIR / 'q4_throttle_optimal.png'}")

```

```
326  
327 if __name__ == "__main__":  
328     main()
```